



UNIVERSIDAD DEL BÍO-BÍO

FACULTAD DE CIENCIAS EMPRESARIALES

Departamento de Sistemas de Información

Ingeniería de Software (620455)

Informe final

Software “CleanAdmin” para empresa de limpieza industrial

Integrantes:

Antonia Peña,

Carlos Bustos,

Fernanda Fernandez,

Javiera Cuevas,

Amasis Guzmán

Profesores:

Alejandra Segura

Sebastian Espinoza

Tabla de Contenidos

Tabla de Contenidos	2
Capítulo I. Introducción	3
I.1 Presentación del contexto	3
I.1.1 Presentación Rubro Empresa	4
Tabla 1. Características de la empresa	4
I.1.2 Descripción de problemas u Oportunidades de Mejora	4
I.2 Propuesta de solución	5
I.3 Composición del Informe	5
Capítulo II. Definición de Software	6
II.1 Objetivo General del Software propuesto	6
II.2 Objetivos Específicos del Software propuesto	6
II.3 Requerimientos Funcionales del software	6
II.3.1 Módulo de Recursos Humanos y Asistencia	6
II.3.2 Módulo de Bodega e Inventario	7
II.3.3 Módulo de Actividades y Calificaciones	7
II.4 Límites del proyecto	7
II.5 Requerimientos No Funcionales del software	7
II.6 Interfaces externas de entrada	8
II.7 Interfaces externas de Salida	8
Capítulo III Análisis y Diseño	9
III.1 Modelo de datos	9
III.2 Actores Casos de uso	15
III.3 Diagrama de casos de uso	16
III.4 Especificación de los Casos de Uso	19
III.5 Diseño interfaz y navegación.	22
III.5.1 Guías de estilos	22
III.5.2 Guía de colores	22
III.5.3 Tipografía	25
III.5.4 Composición de las interfaces	26
Capítulo IV. Desarrollo	33
IV.1 Diseño de arquitectura	33
IV.2 Estructura del código	34
IV.2.1 Backend	34
IV.2.2 Frontend	35
Capítulo V Conclusión del proyecto	36
Capítulo VI Bibliografía	37
Anexo Pruebas de API:	37

Capítulo I. Introducción

I.1 Presentación del contexto

En la actualidad, la transformación digital se ha convertido en un factor determinante para la competitividad de las organizaciones, independientemente del rubro en que operen. Sin embargo, en sectores de servicios intensivos en mano de obra, como el de aseo y limpieza industrial, la adopción de herramientas tecnológicas suele ser más lenta que en otras industrias, debido a la naturaleza descentralizada de sus operaciones y a la percepción de que la digitalización de procesos administrativos no constituye una prioridad frente a la ejecución operativa diaria.

No obstante, esta brecha tecnológica genera consecuencias directas sobre la eficiencia, la trazabilidad y el cumplimiento normativo de las empresas del sector, particularmente en la gestión del capital humano, el control de inventarios distribuidos entre múltiples proyectos y la coordinación de tareas operativas que, en muchos casos, involucran riesgos laborales y exigencias de calificación técnica. Frente a este escenario, surge la necesidad de desarrollar soluciones de software a medida que permitan centralizar la información, automatizar procesos manuales propensos a errores y establecer mecanismos de control y auditoría que resguarden tanto la operación como la seguridad de los trabajadores.

En este contexto se enmarca el presente proyecto, desarrollado en el marco de la asignatura de Ingeniería de Software, cuyo propósito es analizar, diseñar y construir un sistema web (denominado **CleanAdmin**) orientado a resolver las principales ineficiencias identificadas en una empresa real del rubro de servicios de aseo y limpieza industrial. A partir de un levantamiento de información sobre el contexto operativo de la empresa, se definieron los requerimientos funcionales y no funcionales que dieron origen a la arquitectura y al diseño del sistema, los cuales se detallan en los capítulos siguientes.

I.1.1 Presentación Rubro Empresa

El presente proyecto se desarrolla para una empresa de servicios de aseo y limpieza, que opera en el sector industrial, de sanitización y mantenimiento de infraestructura. El sector productivo al que está orientado el proyecto, se caracteriza por la alta intensidad de mano de obra y de operaciones descentralizadas, donde las empresas proveedoras deben desplegar múltiples equipos de trabajo en diversas instalaciones de clientes (fábricas, oficinas, centros comerciales, instituciones de salud o educacionales, entre otros).

Características de la empresa

Tabla 1. Características de la empresa

Aspecto	Descripción
Rubro	Servicios de aseo y limpieza industrial.
Personal	Empleados operativos, supervisores, jefes de proyecto, administradores
Infraestructura	Recursos humanos y bodega para los implementos.
Regulaciones	Normativas sanitarias y contrato con ACHS (prevención de riesgos)
Alcance	Proyectos en diversos tipos de instalaciones, como universidades, instituciones de salud, oficinas, entre otros.

Observación: La empresa gestiona distintos proyectos simultáneamente, cada uno con diferentes requerimientos de personal, recursos e implementos.

I.1.2 Descripción de problemas u Oportunidades de Mejora

A pesar de la relevancia del sector, las empresas que coordinan estas operaciones suelen enfrentarse a ineficiencias críticas debido a la falta de herramientas tecnológicas integradas, lo que da origen a las siguientes problemáticas:

- **Falta de control real y fraudes en la asistencia:** Al trabajar en dependencias externas, la verificación de la asistencia presencial es compleja. Los métodos tradicionales basados en firmas en papel son vulnerables al fraude (marcaje por terceros) y carecen de auditorías inmediatas. Además, el cálculo manual de atrasos, inasistencias injustificadas y la gestión dinámica de los tiempos de colación generan una sobrecarga administrativa propensa al error humano.
- **Falta de trazabilidad y mermas en inventario:** La dispersión de múltiples bodegas satélites en cada proyecto dificulta conocer el stock real. Se pueden generar discrepancias entre los movimientos registrados y las revisiones físicas.
- **Descoordinación en la asignación de tareas operativas:** La planificación de actividades diarias suele ser reactiva. No existen mecanismos automatizados que impidan asignar una tarea especializada o peligrosa a un empleado que no posea la calificación adecuada para ejecutarla, poniendo en riesgo el cumplimiento normativo. Asimismo, ante bajas imprevistas de personal, la reasignación de tareas y la validación de coberturas horarias mínimas se vuelven lentas e ineficientes.
- **Estructura de gobernanza difusa:** La ausencia de un sistema con permisos de acceso granulares expone la información confidencial de la empresa y permite que usuarios no autorizados alteren planificaciones, asignaciones de turnos o registros históricos de auditoría.

I.2 Propuesta de solución

Para mitigar estas ineficiencias, se propone el desarrollo de **CleanAdmin**: un sistema web integral de gestión operativa y control administrativo desarrollado bajo una arquitectura cliente-servidor. La solución centraliza el control de capital humano mediante tokens localizados por IP, automatiza la trazabilidad del inventario, y optimiza la asignación de tareas operativas condicionadas a la calificación técnica del personal. Todo el ecosistema de la plataforma se rige bajo un modelo estricto basado en roles jerárquicos (RBAC).

I.3 Composición del Informe

El presente documento se estructura de la siguiente manera para detallar la ejecución del proyecto:

- Capítulo II (Definición de Software): Describe los objetivos, requerimientos

funcionales y no funcionales, así como los límites del sistema.

- Capítulo III (Análisis y Diseño): Detalla el modelado de datos, los casos de uso y la propuesta estética de las interfaces.
- Capítulo IV (Desarrollo): Explica la arquitectura técnica y la organización del código tanto en el Backend como en el Frontend.
- Capítulo V y VI: Presenta las conclusiones del trabajo realizado y las referencias bibliográficas utilizadas.
- Anexos: Incluye evidencias de pruebas de API y datos recopilados durante el levantamiento de información.

Capítulo II. Definición de Software

II.1 Objetivo General del Software propuesto

Desarrollar e implementar una plataforma tecnológica integrada para la gestión de servicios, que automatice el control de asistencia en tiempo real, optimice los flujos logísticos de inventario distribuido y coordine de manera inteligente la asignación de tareas operativas y calificaciones del personal, para reducir los errores dados por el manejo manual de la información y para optimizar recursos.

II.2 Objetivos Específicos del Software propuesto

Módulo RRHH: Implementar un sistema de marcaje de asistencia mediante códigos QR y tokens alfanuméricos con validación por proximidad de IP, para reducir el fraude laboral y eliminar el cálculo manual de incidencias.

Módulo Bodega: Diseñar un flujo totalmente trazable de solicitudes, ingresos y consumos de insumos (fungibles y herramientas de préstamo) por cada proyecto independiente, para así disminuir las mermas operativas.

Módulo Actividades: Desarrollar un motor de programación y asignación de tareas diarias que valide de forma restrictiva las competencias del empleado antes de permitir la ejecución de actividades industriales o de riesgo.

Jerarquía: Configurar permisos basándose en roles para auditar de punta a punta cualquier alteración del sistema.

II.3 Requerimientos Funcionales del software

II.3.1 Módulo de Recursos Humanos y Asistencia

- **RF-RRHH-01 (Generación de Token):** El sistema debe permitir al Encargado generar un código QR y un token único para un turno activo, capturando un snapshot (captura instantánea de estado) de los empleados asignados a dicho turno.
- **RF-RRHH-06 (Registro de Asistencia):** El sistema permitirá al empleado registrar su entrada ingresando el token respectivo, validando que se encuentre dentro de la ventana de tiempo estipulada y bajo el radio de la IP pública autorizada para el proyecto.

II.3.2 Módulo de Bodega e Inventario

- **RF-BOD-03 (Solicitud de Ítem):** El Encargado en terreno podrá elevar solicitudes de movimiento de stock catalogadas como "SOLICITUD" para ítems no registrados o faltantes en la bodega, las cuales requerirán aprobación superior.

II.3.3 Módulo de Actividades y Calificaciones

- **RF-ACT-04 (Asignación Restrictiva):** El sistema impedirá de manera mandatoria que se asigne una tarea de una categoría específica que requiera calificación a un empleado que no posea su correspondiente registro de calificación activo en la plataforma.

II.4 Límites del proyecto

1. El sistema no realizará liquidaciones de sueldo ni procesará transacciones financieras directamente; su alcance se limita a proveer la reportería de inasistencias y atrasos en un formato auditable.
2. La geolocalización está estrictamente limitada a la estimación por IP pública provista por el navegador del dispositivo; el sistema no realiza rastreo en segundo plano mediante GPS nativo continuo.
3. No se contempla la autogestión o transferencia directa de inventarios compartidos entre proyectos distintos sin la aprobación expresa de un Supervisor mediante una orden de abastecimiento central.

II.5 Requerimientos No Funcionales del software

Seguridad y Privacidad: Almacenamiento seguro de credenciales de acceso mediante algoritmos de hashing robustos (como **bcrypt**) y control estricto de accesos granulares por token.

Disponibilidad y Scheduler: El sistema debe incorporar un servicio de tareas programadas (*Cron/Scheduler*) de alta disponibilidad para ejecutar de forma automatizada y síncrona el cierre de asistencias al finalizar cada turno.

Integridad de Datos: Restricción estricta de borrado a través de la implementación de *Soft Delete* (Borrado Lógico) utilizando el atributo **ACTIVO = False** en las tablas clave. Queda estrictamente prohibido el borrado físico (**DELETE**) de históricos operacionales.

II.6 Interfaces externas de entrada

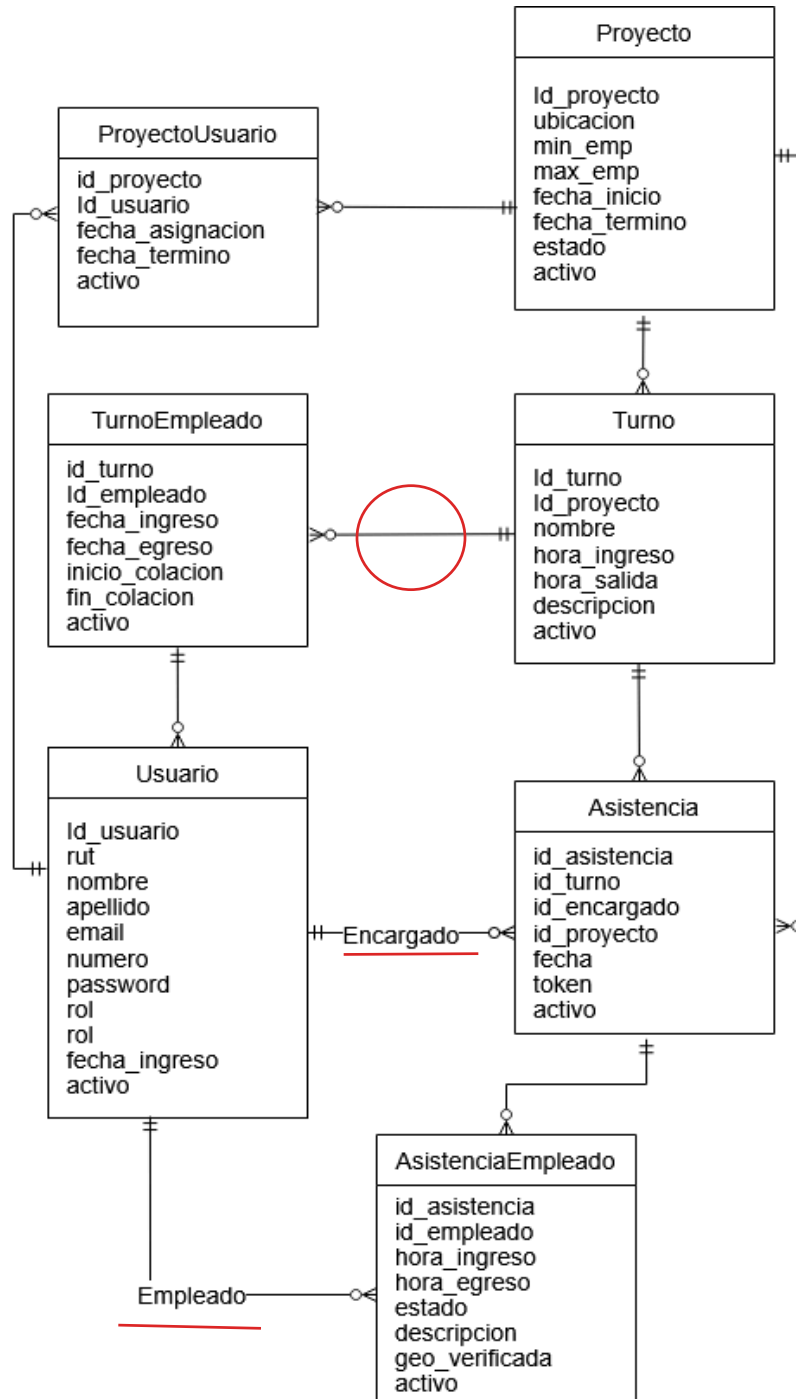
- **Formularios de registro:** Ingreso de datos de los proyectos, empleados, actividades, ítems y turnos.
- **Tokens y QR (para empleados):** Registro de asistencia mediante escaneo o ingreso de código.
- **Selección de archivos:** Interfaz para carga de documentos e informes (opcional).
- Autenticación JWT para ingreso de credenciales (nombre de usuario y contraseña) y búsqueda JSON.

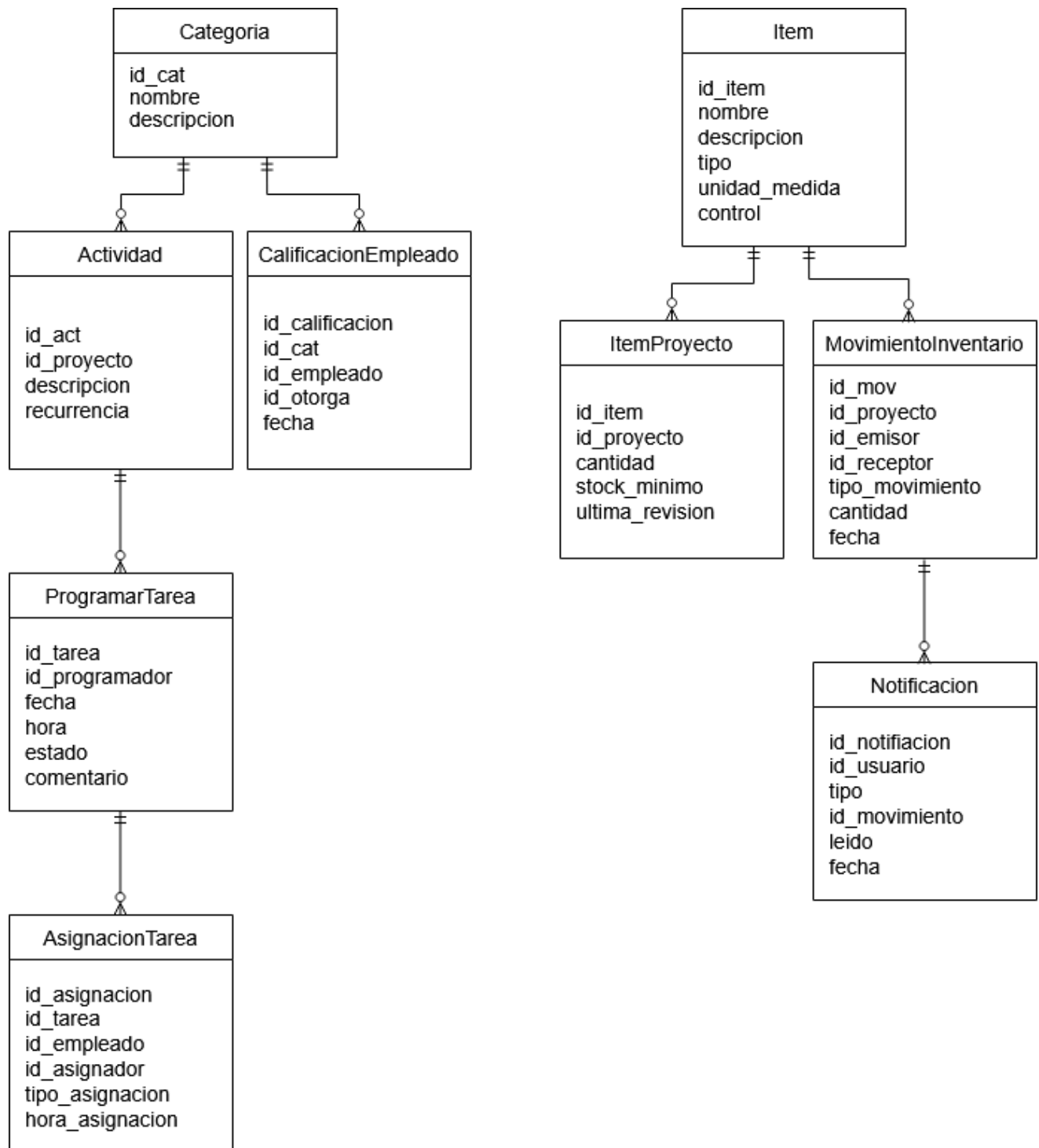
II.7 Interfaces externas de Salida

- **Dashboards visuales:** Paneles de control de proyectos, personal y actividades.
- **Generador de PDFs:** Exportación opcional de reportes de asistencia, actividades, proyectos y turnos.
- **Historiales:** Visualización de registros históricos por empleado, proyecto o actividad.
- **Sistema de notificaciones:** Alertas enviadas al crear un nuevo ítem.

Capítulo III Análisis y Diseño

III.1 Modelo de datos





1. Usuario

- a. id_usuario (int): Identificador único autogenerated (PK).
- b. rut (varchar 15): RUT del usuario, único en el sistema.
- c. nombre (varchar 100): Nombre del usuario.
- d. apellido (varchar 100): Apellido del usuario.
- e. email (varchar 100): Correo electrónico.
- f. numero (varchar 15): Número de teléfono, único.
- g. password (varchar 255): Hash de contraseña; excluido de consultas SELECT.
- h. rol (varchar 100): Rol del usuario: ROOT, ADMIN, SUPERVISOR, ENCARGADO, EMPLEADO.
- i. observacion (text): Observaciones sobre el usuario.
- j. fecha_ingreso (timestamp): Fecha de creación del registro (default: ahora).
- k. activo (boolean): Indica si el usuario está habilitado en el sistema

2. Proyecto.

- a. id_proyecto (int): Identificador único autogenerated (PK).
- b. nombre_proy (varchar 100): Nombre descriptivo del proyecto.
- c. min_emp (int): Mínimo de empleados requeridos.
- d. max_emp (int): Máximo de empleados permitidos.
- e. ubicacion (text): Ubicación geográfica del proyecto.
- f. fecha_inicio (timestamp): Fecha de inicio del proyecto.
- g. fecha_termino (timestamp, nullable): Fecha de término; null si sigue activo.
- h. estado (enum): Estado operativo: EN_PREPARACION, EN_CURSO, FINALIZADO.
- i. activo (boolean): Indica si el proyecto está vigente.

3. Item

- a. id_item (int): Identificador único autogenerated (PK).
- b. nombre (varchar 255): Nombre del ítem, único en el sistema.
- c. descripcion (text, nullable): Descripción detallada del recurso.
- d. tipo (enum): Clasificación: MAQUINARIA, HERRAMIENTA, UTENSILIO, PRODUCTO.
- e. unidad_medida (enum): Unidad: LITROS, UNIDADES, KILOS, SACOS, BOLSAS, METROS.
- f. control (enum): Método de gestión: CONSUMO, PRESTAMO.
- g. activo (boolean): Indica si el ítem está disponible.

4. Asistencia

- a. id_asistencia (int): Identificador único autogenerated (PK).
- b. fecha (date): Fecha de la sesión de asistencia.
- c. token (varchar 4): Código alfanumérico único para validación en terreno.
- d. token_expira (timestamp): Fecha y hora de expiración del token.
- e. activo (boolean): Indica si la sesión está abierta.

- f. id_turno (FK → Turno): Turno asociado.
- g. id_encargado (FK → Usuario): Encargado que creó la sesión.
- h. id_proyecto (FK → Proyecto): Proyecto al que pertenece.

5. AsistenciaEmpleado

- a. id_asistencia (int, PK compuesta, FK → Asistencia): Sesión de asistencia.
- b. id_empleado (int, PK compuesta, FK → Usuario): Empleado registrado.
- c. hora_ingreso (time, nullable): Hora de entrada registrada.
- d. hora_egreso (time, nullable): Hora de salida registrada.
- e. estado (enum): PRESENTE, EN_ESPERA, RETIRADO, ATRASO, FALTA_JUSTIFICADA, FALTA_INJUSTIFICADA.
- f. descripcion (text, nullable): Observaciones o justificaciones.
- g. geo_verificada (boolean): Indica si la asistencia fue verificada por geolocalización.
- h. editado_por (int, nullable): ID del usuario que realizó una edición posterior.
- i. fecha_edicion (timestamp, nullable): Fecha de la última edición

6. Turno

- a. id_turno (int): Identificador único autogenerado (PK).
- b. nombre (varchar 100): Nombre descriptivo del turno.
- c. hora_ingreso (time): Hora oficial de entrada.
- d. hora_salida (time): Hora oficial de salida.
- e. descripcion (text, nullable): Información adicional del turno.
- f. activo (boolean): Estado de vigencia del turno.
- g. id_proyecto (FK → Proyecto): Proyecto al que pertenece el turno.

7. TurnoEmpleado

- a. id_turno (int, PK compuesta, FK → Turno): Turno asignado.
- b. id_empleado (int, PK compuesta, FK → Usuario): Empleado asignado.
- c. fecha_ingreso (date): Inicio de la asignación al turno.
- d. fecha_egreso (date, nullable): Fin de la asignación; null si sigue vigente.
- e. inicio_colacion (time, nullable): Inicio del período de colación.
- f. fin_colacion (time, nullable): Fin del período de colación.
- g. trabaja_feridos (boolean): Indica si el empleado trabaja en días feriados.
- h. activo (boolean): Estado de la asignación.

8. Categoria

- a. id_cat (int): Identificador único autogenerado (PK).
- b. nombre (varchar 100): Nombre de la categoría, único en el sistema.
- c. descripcion (text, nullable): Descripción de la categoría.
- d. requiere_calificacion (boolean): Indica si los empleados deben estar calificados para ejecutar actividades de esta categoría.
- e. activo (boolean): Estado de la categoría.

9. Actividad



- a. id_act (int): Identificador único autogenerated (PK).
- b. descripcion_esp (text): Descripción específica de la actividad.
- c. recurrencia (enum): DIARIA, SEMANAL, MENSUAL, UNICA.
- d. activo (boolean): Estado de la actividad.
- e. id_cat (FK → Categoría): Categoría a la que pertenece.
- f. id_proyecto (FK → Proyecto): Proyecto al que está asociada.

10. ProgramarTarea

- a. id_tarea (int): Identificador único autogenerated (PK).
- b. fecha (date): Fecha de ejecución programada.
- c. hora (time): Hora de ejecución.
- d. estado (enum): PLANIFICADA, EN_PROCESO, FINALIZADA, INCOMPLETA, CANCELADA.
- e. comentario (varchar 255, nullable): Observación del encargado o supervisor.
- f. id_act (FK → Actividad): Actividad base de la tarea.
- g. id_programador (FK → Usuario): Usuario que programó la tarea.

11. AsignacionTarea

- a. id_asignacion (int): Identificador único autogenerated (PK).
- b. tipo_asignacion (enum): PROGRAMADA, REASIGNADA.
- c. hora_asignacion (timestamp): Momento en que se realizó la asignación.
- d. id_tarea (FK → ProgramarTarea): Tarea asignada.
- e. id_empleado (FK → Usuario): Empleado asignado.
- f. id_asignador (FK → Usuario): Usuario que realizó la asignación.

12. CalificacionEmpleado

- a. id_calificacion (int): Identificador único autogenerated (PK).
- b. fecha_otorgamiento (timestamp): Fecha en que se otorgó la calificación.
- c. activo (boolean): Estado de la calificación.
- d. id_cat (FK → Categoría): Categoría para la que se otorga la calificación.
- e. id_empleado (FK → Usuario): Empleado calificado.
- f. id_otorga (FK → Usuario): Usuario que otorgó la calificación.

13. ItemProyecto

- a. id_item (int, PK compuesta, FK → Ítem): Ítem asociado.
- b. id_proyecto (int, PK compuesta, FK → Proyecto): Proyecto asociado.
- c. cantidad (int): Cantidad disponible en el proyecto.
- d. stock_minimo (int): Nivel mínimo de stock que activa alertas.
- e. ultima_revision (timestamp, nullable): Fecha de la última revisión de inventario.
- f. activo (boolean): Estado de la asociación.

14. MovimientoInventario

- a. id_mov (int): Identificador único autogenerated (PK).
- b. id_proyecto (int): Proyecto en el que ocurre el movimiento.
- c. id_emisor (int): Usuario que emite el movimiento.

- d. id_receptor (int, nullable): Usuario receptor (en préstamos).
- e. tipo_movimiento (enum): ENTRADA, SALIDA, SOLICITUD, ABASTECIMIENTO, COMPRA.
- f. cantidad (int): Cantidad involucrada en el movimiento.
- g. fecha (timestamp): Fecha y hora del movimiento.
- h. descripcion (text, nullable): Detalle opcional del movimiento.
- i. estado_solicitud (enum, nullable): PENDIENTE, APROBADO, RECHAZADO (para SOLICITUD).
- j. item_sugerido (varchar 100, nullable): Nombre sugerido si el ítem no existe aún.
- k. id_item (FK → Ítem, nullable): Ítem involucrado.

15. ProyectoUsuario

- a. id_proyecto (int, PK compuesta, FK → Proyecto): Proyecto.
- b. id_usuario (int, PK compuesta, FK → Usuario): Usuario asignado.
- c. fecha_asignacion (timestamp): Fecha de inicio de la asignación.
- d. fecha_termino (timestamp, nullable): Fecha de fin; null si sigue activo.
- e. activo (boolean): Estado de la asignación.

16. Notificacion

- a. id_notificacion (int): Identificador único autogenerado (PK).
- b. id_usuario_destinatario (int): Usuario al que va dirigida la notificación.
- c. tipo (varchar 50): Tipo: SOLICITUD_PENDIENTE, SOLICITUD_APROBADA, SOLICITUD_RECHAZADA.
- d. id_movimiento (int): Movimiento de inventario que originó la notificación.
- e. leído (boolean): Indica si la notificación fue leída.
- f. fecha (timestamp): Fecha y hora de creación

Relaciones entre entidades

Entidad Origen	Entidad Destino	Tipo de Relación	Descripción
Usuario	Proyecto	M:N (via ProyectoUsuario)	Un usuario puede participar en múltiples proyectos y viceversa, con control de fechas de asignación.
Usuario	Turno	M:N (via TurnoEmpleado)	Un empleado puede pertenecer a varios turnos; un turno agrupa a múltiples empleados.
Proyecto	Turno	1:N	Un proyecto puede tener múltiples turnos definidos.
Proyecto	Actividad	1:N	Un proyecto contiene múltiples actividades operativas.

Proyecto	Ítem	M:N (via ItemProyecto)	Un proyecto puede gestionar múltiples ítems y un ítem puede estar en varios proyectos.
Asistencia	Turno	N:1	Cada sesión de asistencia está vinculada a un turno específico.
Asistencia	Usuario (encargado)	N:1	Un encargado puede crear múltiples sesiones de asistencia.
Asistencia	AsistenciaEmpleado	1:N	Una sesión de asistencia agrupa el registro individual de cada empleado.
Actividad	Categoría	N:1	Cada actividad pertenece a una categoría que puede requerir calificación.
ProgramarTarea	Actividad	N:1	Una tarea programada es una instancia de una actividad base.
ProgramarTarea	AsignacionTarea	1:N	Una tarea puede tener múltiples asignaciones (incluyendo reasignaciones).
CalificacionEmpleado	Usuario	N:1 (empleado)	Registra la calificación de un empleado en una categoría específica.
Ítem	MovimientoInventario	1:N	Un ítem puede tener múltiples movimientos de entrada, salida, préstamo o solicitud.
MovimientoInventario	Notificacion	1:N	Un movimiento tipo SOLICITUD genera notificaciones para supervisores y administradores.

III.2 Actores Casos de uso

Actor	Cargo(s)	Nivel de Conocimientos Técnicos	Nivel de Privilegio en el Sistema
Root	Administrador de Sistema	Muy Alto	Control total e irrevocable del sistema. Gestiona a otros administradores y configura permisos globales.
Administrador	Gestión de Proyectos	Alto	Gestión global de proyectos, presupuestos de personal y asignación de permisos. Aprueba solicitudes de inventario.
Supervisor	Control de Personal	Medio	Validación, auditoría, aprobación de inventario, creación de categorías y control de múltiples proyectos.
Encargado	Operaciones en Terreno	Bajo-Medio	Operación diaria en terreno: control de asistencia mediante token/QR, asignación de tareas diarias y solicitudes de stock.
Empleado	Operario	Bajo	Vista de turnos propios, ejecución y reporte de tareas asignadas, y registro de asistencia

Actor	Cargo(s)	Nivel de Conocimientos Técnicos	Nivel de Privilegio en el Sistema
mediante token o QR.			

Descripción de Actores

1. **Root:** Administrador absoluto e irrevocable del sistema. Puede crear, editar y eliminar cuentas de administradores, asignar permisos globales, y tiene visibilidad total sobre todos los módulos del sistema sin restricciones.
2. **Administrador:** Responsable de la gestión integral de proyectos. Puede crear y administrar proyectos, asignar empleados, supervisores y encargados, gestionar el inventario y aprobar solicitudes de materiales emitidas por los encargados de terreno.
3. **Supervisor:** Encargado del control y seguimiento de uno o más proyectos. Puede validar actividades, crear y gestionar categorías, revisar el estado de inventario, aprobar o rechazar solicitudes de stock y auditar los registros de asistencia.
4. **Encargado:** Opera directamente en terreno dentro de un proyecto asignado. Sus funciones principales son generar tokens/QR para el registro de asistencia, asignar tareas diarias a empleados y realizar solicitudes de materiales a bodega.
5. **Empleado:** Usuario de nivel operativo. Registra su asistencia mediante el token o código QR generado por el encargado, consulta sus turnos y tareas asignadas, y reporta el progreso o finalización de sus actividades.

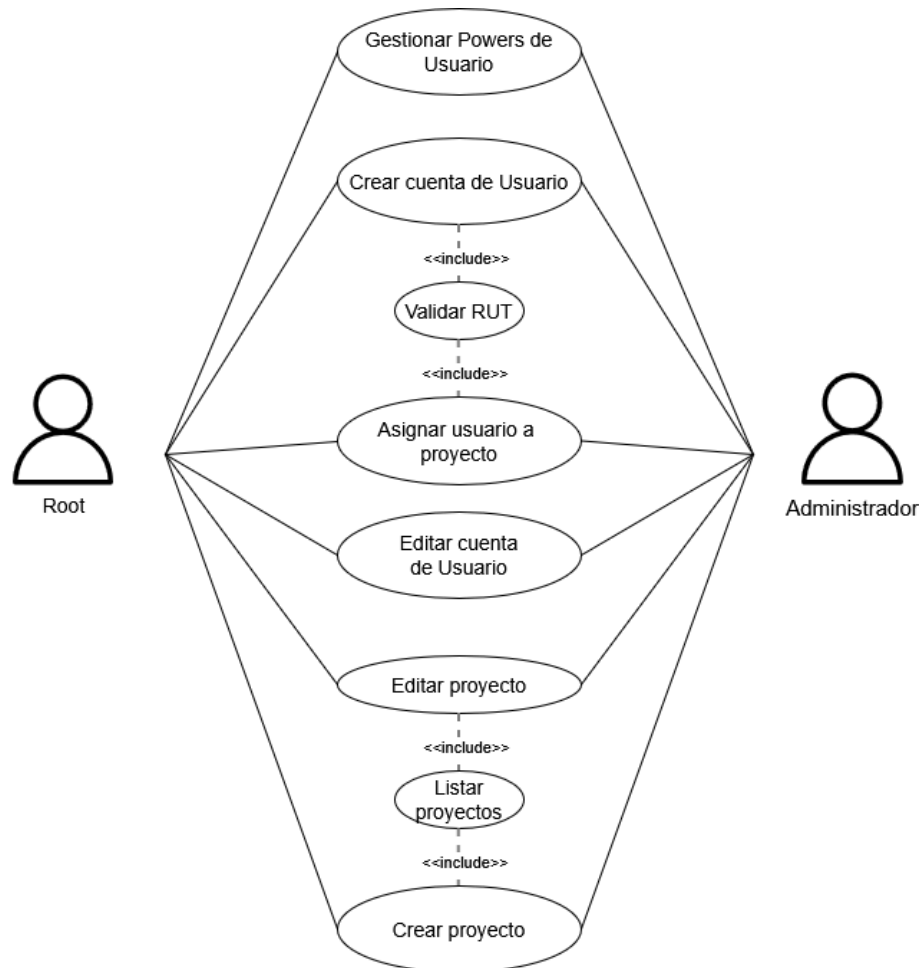
III.3 Diagrama de casos de uso

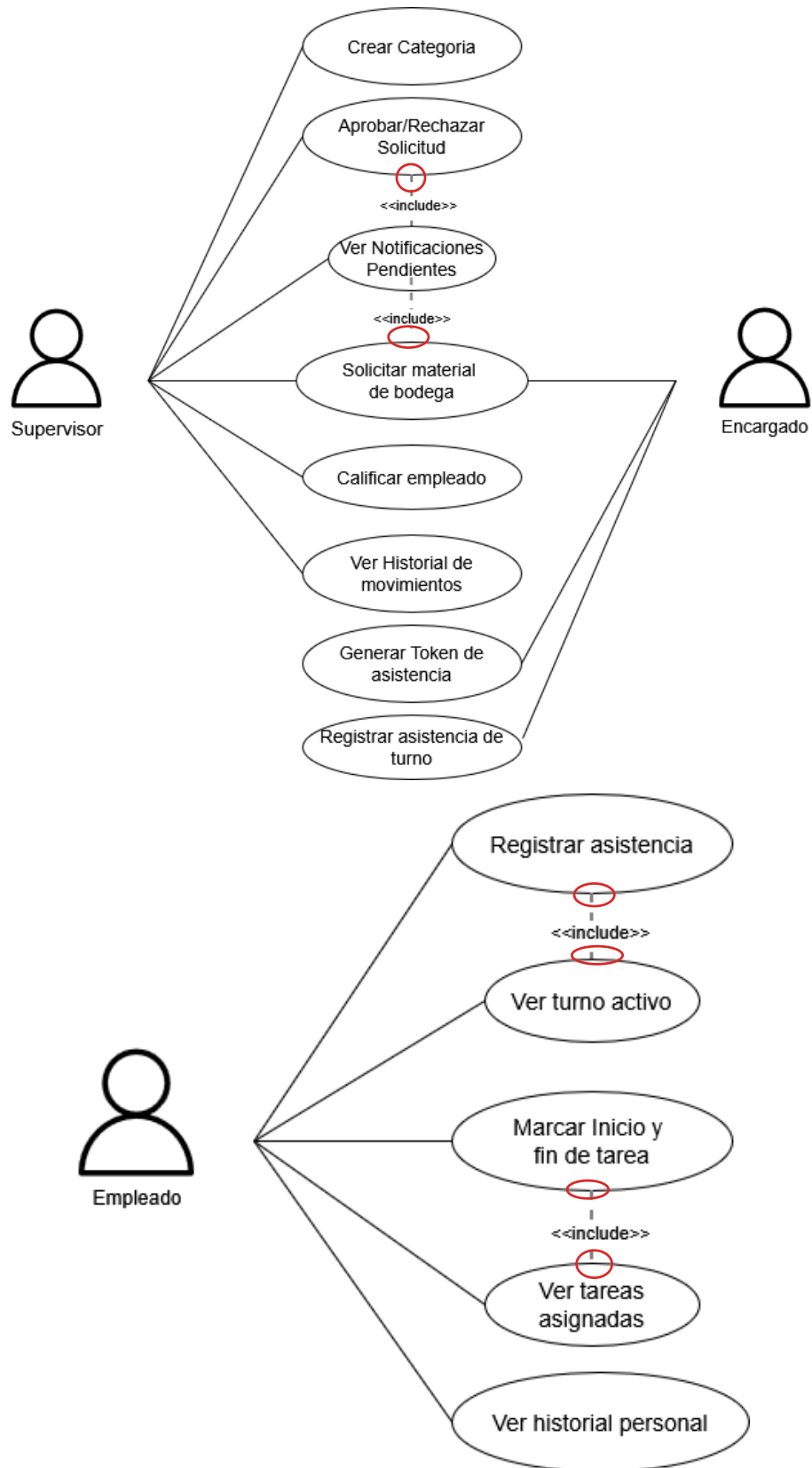
Lista de Casos de Uso

Casos de uso con especificación	Casos de uso con descripción general
1. Crear proyecto.	1. Validar RUT.
2. Registrar asistencia con token/QR.	2. Listar proyectos por estado.
3. Asignar ítem a proyecto.	3. Ver historial de asistencia.
4. Crear cuenta de usuario.	4. Ver estadísticas de personal.
5. Programar tarea.	5. Filtrar ítems por tipo y control.
6. Asignar tarea a empleado.	6. Generar token/QR de asistencia.
7. Solicitar material de bodega.	7. Ver calificaciones de empleado.
8. Aprobar/rechazar solicitud de inventario	8. Otorgar calificación a empleado.
	9. Editar registro de asistencia.

10. Ver notificaciones pendientes.

11. Crear turno y asignar empleados.





III.4 Especificación de los Casos de Uso

Tabla 1: CU Crear proyecto.

CU: CREAR PROYECTO	
Precondiciones:	El usuario debe estar autenticado con rol ROOT o ADMINISTRADOR.
Flujo de acciones:	1. El sistema despliega el formulario de creación de proyecto. 2. El administrador ingresa nombre del proyecto, ubicación, fechas de inicio y término, estado inicial (EN_PREPARACION) y límites de personal (min_emp, max_emp). 3. El sistema valida que min_emp sea menor o igual a max_emp y que los campos obligatorios estén completos. 4. Si la validación es correcta, el proyecto se guarda en la base de datos; de lo contrario, se devuelve al paso 2 con indicación del error. 5. El sistema retorna confirmación de creación y muestra el proyecto en el listado.
Postcondiciones:	Se registra el nuevo proyecto en la base de datos con estado EN_PREPARACION y activo = true.

Tabla 2: CU Registrar asistencia con token/QR.

CU: REGISTRAR ASISTENCIA CON TOKEN/QR	
Precondiciones:	El empleado debe estar autenticado. Debe existir una sesión de asistencia activa con token vigente (no expirado), generada por el encargado del proyecto.
Include:	CU Generar token/QR de asistencia.
Flujo de acciones:	1. El encargado crea una sesión de asistencia para el turno y proyecto, generando un token de 4 caracteres con expiración definida. 2. El empleado accede a la vista de asistencia e ingresa el token manualmente o escanea el código QR. 3. El sistema verifica que el token exista, que no haya expirado (token_expira) y que pertenezca al proyecto del empleado. 4. Si el token es válido, se crea o actualiza el registro en AsistenciaEmpleado con estado PRESENTE y hora de ingreso. 5. Si el token es inválido o ha expirado, se notifica al empleado con el error correspondiente.
Postcondiciones:	Se registra la asistencia del empleado en la base de datos con estado PRESENTE y la hora de ingreso actual.

Tabla 3: CU Programar tarea.

CU: PROGRAMAR TAREA	
Precondiciones:	El usuario debe estar autenticado con rol ROOT, ADMINISTRADOR, SUPERVISOR o ENCARGADO. La actividad base y el proyecto deben existir y estar activos.
Flujo de acciones:	1. El usuario selecciona el proyecto y accede al módulo de tareas. 2. El sistema despliega las actividades disponibles del proyecto con sus categorías. 3. El usuario selecciona la actividad, define la fecha, hora y agrega un comentario opcional. 4. El sistema verifica que la fecha sea futura y que la actividad esté activa. 5. Si la validación es correcta, se crea la tarea programada con estado PLANIFICADA. 6. Si hay error de validación, se regresa al paso 3 con el mensaje correspondiente.
Postcondiciones:	Se registra la tarea programada en la base de datos con estado PLANIFICADA, lista para ser asignada a empleados.

Tabla 4: CU Asignar tarea a empleado.

CU: ASIGNAR TAREA A EMPLEADO	
Precondiciones:	El usuario debe estar autenticado con rol ROOT, ADMINISTRADOR, SUPERVISOR o ENCARGADO. Debe existir una tarea programada en estado PLANIFICADA y empleados asignados al proyecto.
Flujo de acciones:	1. El usuario accede a la tarea programada y selecciona la opción de asignar. 2. El sistema despliega la lista de empleados asignados al proyecto con sus turnos activos. 3. Si la actividad pertenece a una categoría con requiere_calificacion = true, el sistema filtra solo los empleados con calificación vigente en esa categoría. 4. El usuario selecciona al empleado y confirma la asignación. 5. El sistema registra la AsignacionTarea con tipo PROGRAMADA y actualiza el estado de la tarea a EN_PROCESO si corresponde.
Postcondiciones:	Se crea el registro de AsignacionTarea vinculando el empleado a la tarea. Si la tarea era la primera asignación, su estado cambia a EN_PROCESO.

Tabla 5: CU Solicitar material de bodega.

CU: SOLICITAR MATERIAL DE BODEGA	
Precondiciones:	El usuario debe estar autenticado con rol ENCARGADO, SUPERVISOR o ADMINISTRADOR y estar asignado al proyecto.
Include:	CU Ver notificaciones pendientes.
Flujo de acciones:	1. El encargado accede al módulo de inventario del proyecto. 2. El sistema despliega el stock disponible de ítems del proyecto (ItemProyecto). 3. El encargado selecciona el ítem, la cantidad requerida y agrega una descripción. 4. El sistema crea un MovimientoInventario con tipo_movimiento = SOLICITUD y estado_solicitud = PENDIENTE. 5. El sistema genera notificaciones para todos los usuarios con rol SUPERVISOR, ADMIN o ROOT del proyecto. 6. El supervisor o administrador puede aprobar o rechazar la solicitud, actualizando el estado_solicitud y el stock.
Postcondiciones:	Se registra la solicitud en MovimientoInventario y se generan notificaciones para los supervisores y administradores del proyecto.

Tabla 6: CU Crear cuenta de usuario.

CU: CREAR CUENTA DE USUARIO	
Precondiciones:	El usuario debe estar autenticado con rol ROOT o ADMINISTRADOR.
Include:	CU Validar RUT.
Flujo de acciones:	1. El sistema despliega el formulario de creación de usuario. 2. El administrador ingresa nombre, apellido, RUT, email, número de teléfono, contraseña y rol. 3. El sistema llama a CU Validar RUT para verificar el formato y unicidad del RUT. 4. El sistema verifica que el email y número de teléfono no estén ya registrados. 5. Si todas las validaciones son correctas, la contraseña se encripta (bcrypt) y los datos se guardan en la base de datos. 6. Si hay un error de validación, se regresa al paso 2 con indicación del campo inválido.
Postcondiciones:	Se crea la cuenta de usuario en la base de datos con la contraseña almacenada como hash bcrypt y activo = true.

III.5 Diseño interfaz y navegación.

El diseño de la interfaz de CleanAdmin se basa en una arquitectura de componentes reutilizables contruidos con React. La estilización se apoya en Tailwind CSS v4 y la implementación de variables CSS personalizadas para la gestión del aspecto visual

La arquitectura de navegación se estructura en dos layouts principales diseñados para responder a las necesidades, flujos de trabajo y niveles de privilegio de los distintos perfiles de usuario:

Layout Principal: Orientado a roles con responsabilidad de gestión administrativa. Se compone de una barra lateral de navegación fija y un panel de contenido principal que incluye una barra superior. Este diseño facilita el acceso simultáneo, panorámico y eficiente a los múltiples módulos del sistema (como proyectos, personal, inventario y categorías).

Layout Operativo: Diseñado exclusivamente para el rol de Empleado. Presenta una interfaz simple y optimizada permitiendo un acceso rápido y directo a las funcionalidades diarias como el registro de asistencia, revisión de turnos y la ejecución de tareas.

III.5.1 Guías de estilos

La interfaz de CleanAdmin adopta un lenguaje visual moderno aplicado de manera uniforme a través de todos los componentes del sistema (tales como tarjetas, modales, botones, tablas y barras de navegación)

III.5.2 Guía de colores

La paleta de colores está definida mediante variables CSS personalizadas (Tailwind CSS 4), lo que permite una personalización flexible del tema visual del sistema.

Colores principales

Define la identidad visual de la aplicación mediante una gama vibrante de tonos violetas y azules. Estos colores se aplican en elementos interactivos de alta prioridad, fondos con gradientes, botones principales y estados activos de navegación, con el objetivo de guiar la atención del usuario y mantener una estética moderna y distintiva.



Es el color principal. Se usa en el gradiente del fondo general, *sidebar*, el fondo de los botones principales, el estado activo de las pestañas y el texto de los botones de turnos.

#7C3AED



Se utiliza para la parte inferior del gradiente de los botones principales, dándoles profundidad

#4C1D95

Utilizado específicamente para el texto de las pestañas cuando están activas.

#6D28D9



Funciona como color de acento y para el brillo de los elementos activos en el menú.

#A78BFA

Se utiliza con distintas opacidades para los fondos al pasar el cursor en las tablas, decoradores de tarjetas y los fondos de los iconos.

#8B5CF6



Color secundario de la marca. Se mezcla con el violeta para crear los gradientes del fondo, del *sidebar* y de los elementos activos.

#3B82F6

Escala de grises y blancos

Agrupada desde tonos casi negros hasta blancos puros, destinada a la estructuración espacial y la jerarquización tipográfica. Por un lado, los blancos y grises extra claros conforman los contenedores base, tarjetas, divisores y bordes, proporcionando un lienzo limpio que facilita la separación de componentes. Por otro lado, la escala de grises de mayor contraste organiza la lectura de títulos, textos regulares, etiquetas y datos tabulares. En conjunto, aseguran una legibilidad óptima, mantienen la limpieza visual y evitan la fatiga durante el uso prolongado del sistema.



Se usa para los títulos principales, el texto primario del sidebar y los números destacados en las tarjetas.

#0F172A



Se usa para el texto regular de lectura, las opciones del menú, las pestañas inactivas y el contenido de las tablas.

#334155

Se usa para texto secundario, subtítulos, descripciones de turnos y las acciones genéricas de las tablas.

#64748B



Se emplea para etiquetas, textos secundarios del menú y el texto cuando una tabla está vacía.

#94A3B8

El color principal para delimitar elementos, como los bordes de las tarjetas, el borde de la tabla y los encabezados.

#E2E8F0



Se utiliza para divisores dentro de las tarjetas y bordes suaves de las tablas.

#F1F5F9

Parte del gradiente del fondo de la aplicación y fondo de las secciones de horarios en los turnos.

#F8FAFC

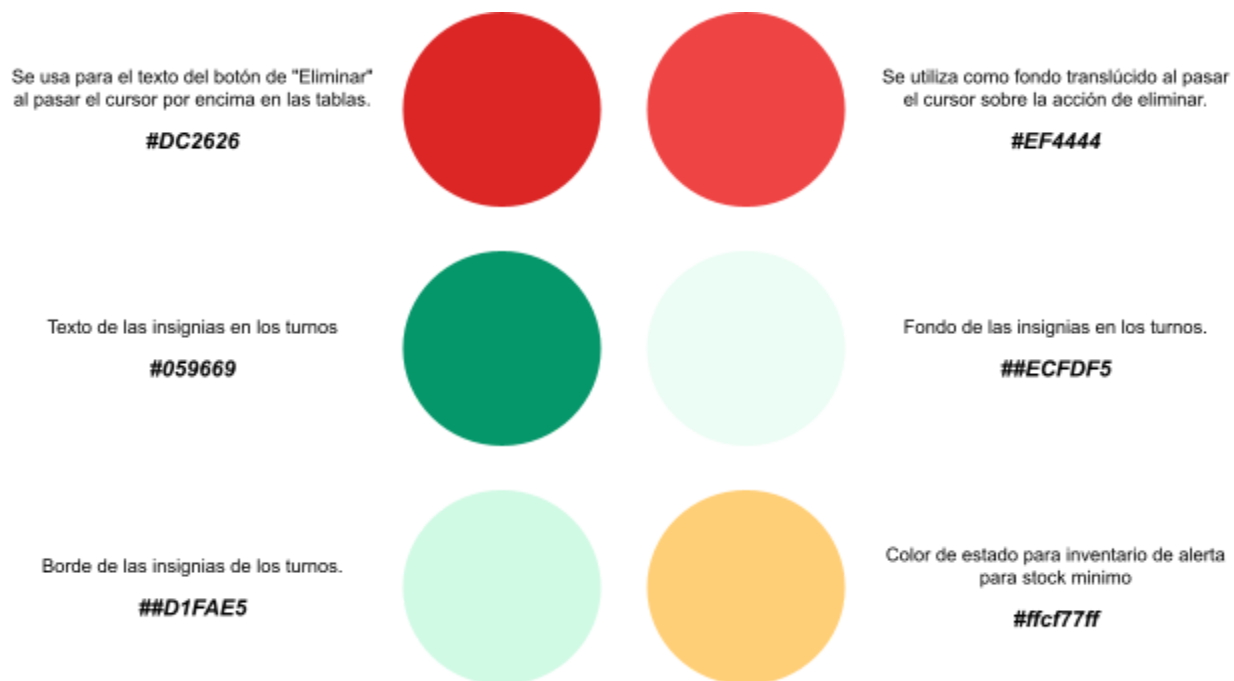


Se utiliza para el fondo de las tarjetas (cards), el fondo general del layout, el fondo del sidebar y las tablas.

#####

Colores para acciones y estados

Estos colores cumplen una función comunicativa y universal, proporcionando retroalimentación inmediata sobre el contexto del sistema. Se emplean gamas de color rojo para advertir sobre acciones destructivas o críticas (como la eliminación de registros) y tonos verdes para confirmar operaciones exitosas o estados positivos (como turnos activos o completados), minimizando así el margen de error operativo.



III.5.3 Tipografía

El sistema utiliza el stack tipográfico por defecto que provee Tailwind CSS v4, framework sobre el cual está construida la interfaz:

ui-sans-serif, system-ui, sans-serif

Esto implica que el texto de la aplicación se renderiza con la tipografía nativa del sistema operativo del usuario final

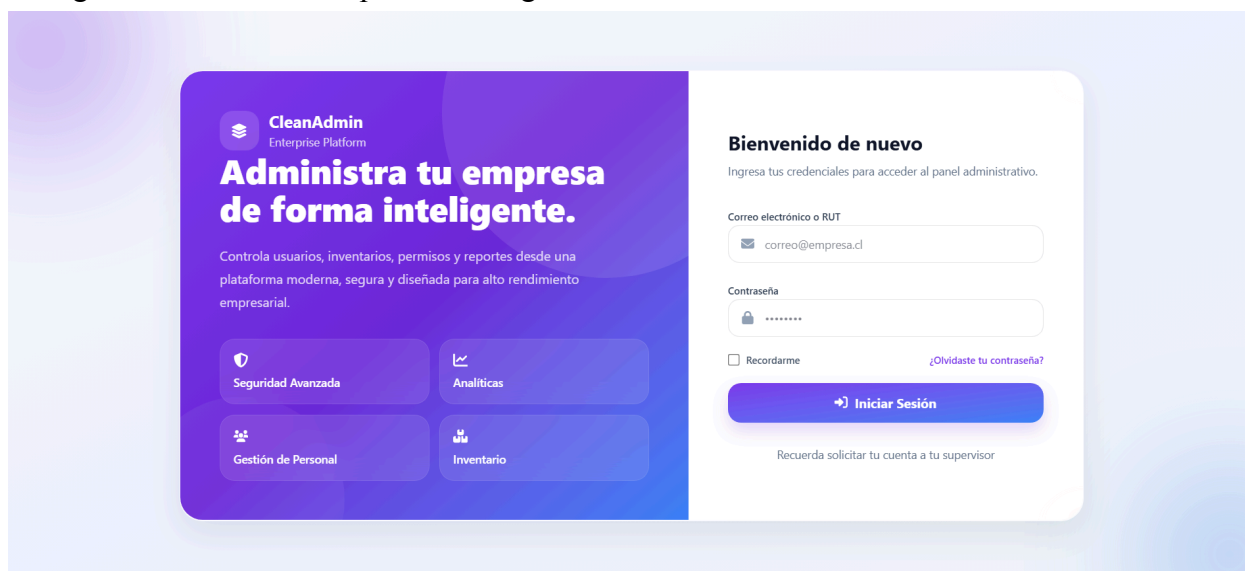
Especificación tipográfica.

Elemento	Tamaño / Peso	Uso
Título de módulo	24px / Bold	Encabezado de cada sección o módulo activo en el TopBar.
Subtítulo / TopBar	14px / Regular	Subtítulo descriptivo del módulo activo.
Cuerpo de texto	14px / Regular	Contenido general, celdas de tablas y descripciones.
Etiqueta de campo	12px / Medium	Labels de inputs y filtros en formularios.

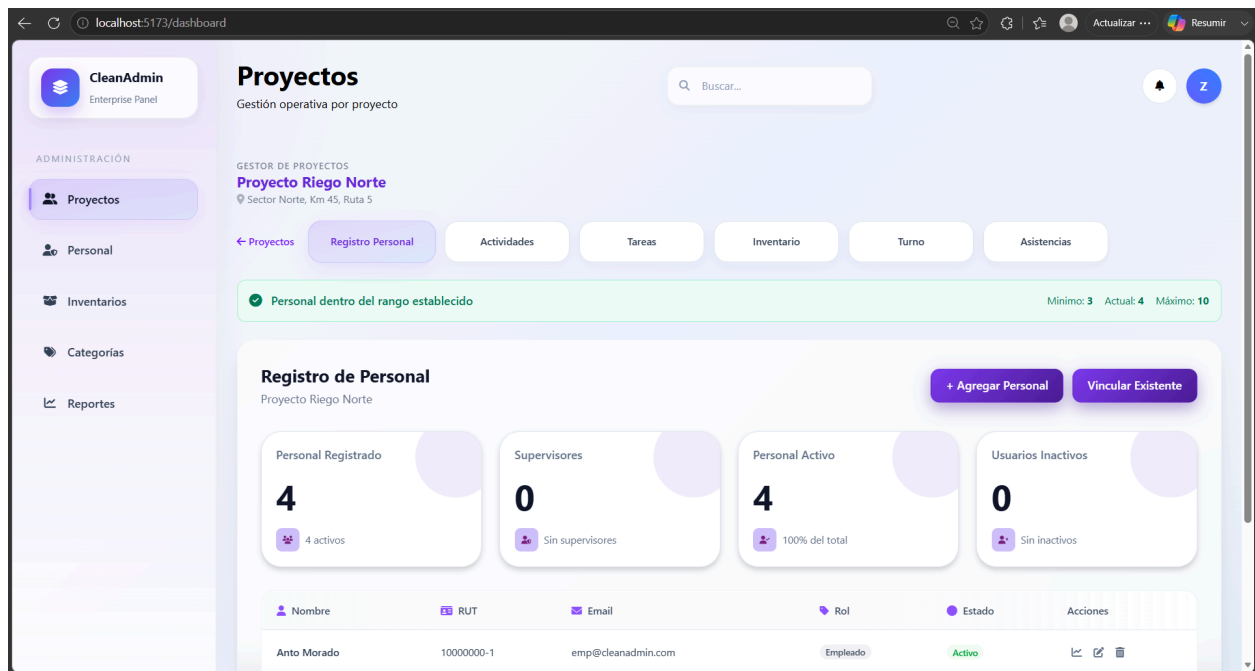
Botón de acción	14px / SemiBold	Texto de botones primarios y secundarios.
Badge / Estado	11px / SemiBold	Etiquetas de estado (EN_CURSO, PRESENTE, APROBADO, etc.).

III.5.4 Composición de las interfaces

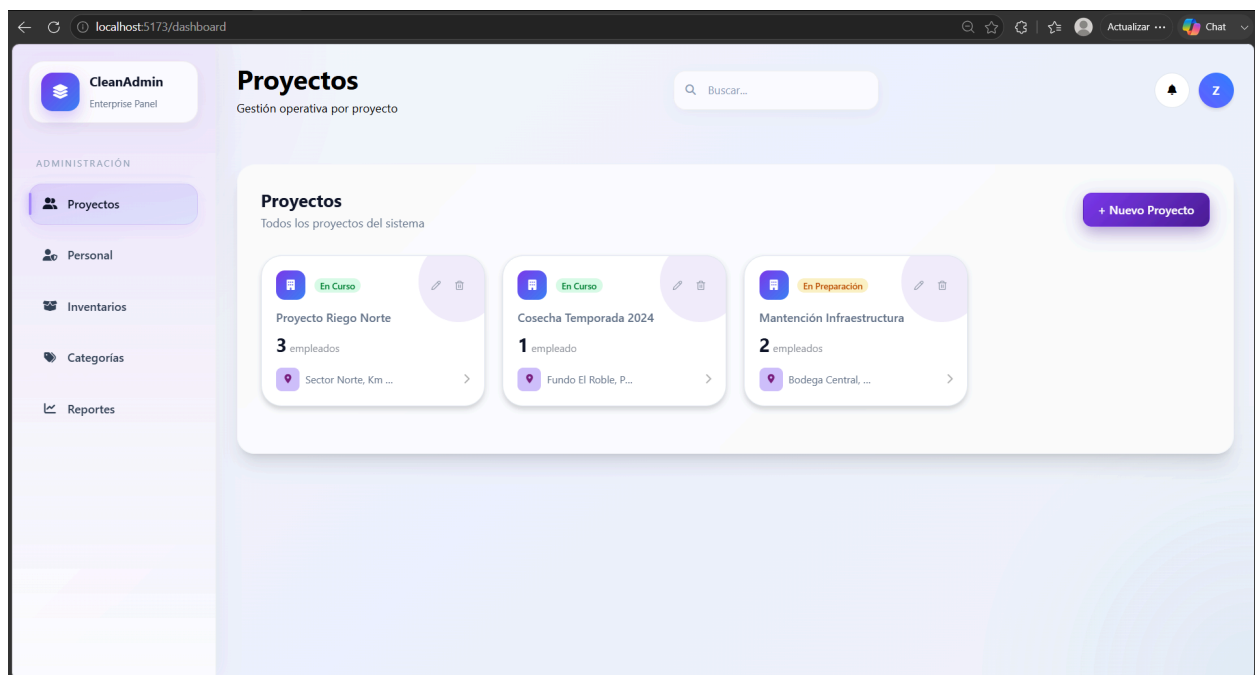
Inicio de sesión (Login): Pantalla de autenticación mediante RUT y contraseña. El sistema redirige al dashboard correspondiente según el rol del usuario autenticado



Dashboard administrativo (AdminPage): Panel principal para roles ROOT, ADMIN, SUPERVISOR y ENCARGADO. Incluye sidebar con acceso a los módulos de Proyectos, Personal, Inventarios y Categorías, más una barra superior con búsqueda y notificaciones.

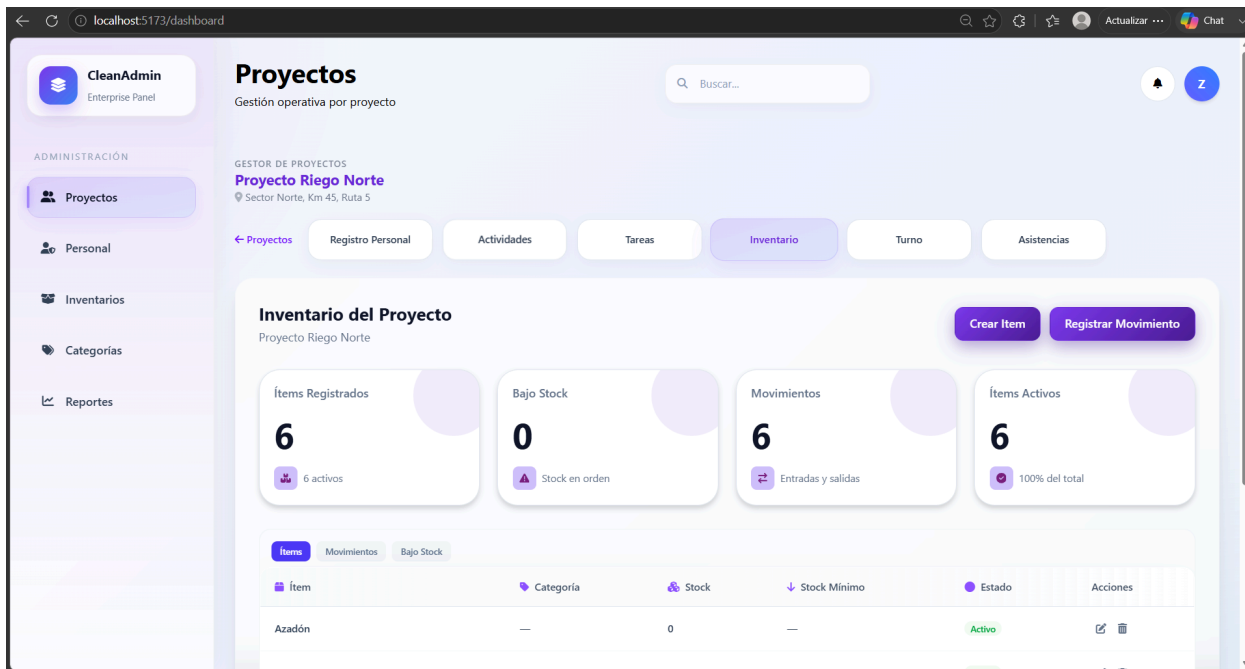


Módulo de Proyectos (ProyectosView): Visualización y gestión de proyectos del sistema. Permite crear, editar y cambiar el estado de cada proyecto, con acceso a sub-módulos de turnos, asistencia, inventario y actividades del proyecto.



Módulo de Personal (PersonalView): Gestión de usuarios del sistema y asignación a proyectos. Incluye tabs para visualización de empleados asignados, sus turnos y calificaciones.

Módulo de Inventarios (InventariosView): Control de stock de ítems por proyecto. Permite gestionar movimientos (ENTRADA, SALIDA, SOLICITUD, COMPRA, ABASTECIMIENTO), aprobar solicitudes y visualizar el historial de movimientos.



Módulo de Categorías (CategoríasView): Gestión de categorías de actividades. Permite crear y administrar categorías con o sin requisito de calificación, y gestionar las calificaciones de empleados por categoría.

←

🔄

🌐 localhost:5173/dashboard

CleanAdmin

Enterprise Panel

Categorías

Gestión del catálogo global

🔍

Buscar...

👤

Actualizar...

🌐

Chat

ADMINISTRACIÓN

👤 Proyectos

👤 Personal

📦 Inventarios

📁 Categorías

📊 Reportes

Gestión de Categorías

Catálogo global de clasificaciones para actividades

Nueva Categoría

Ver Inactivas

Categorías Activas

3

📁 Catálogo global

Requieren Calificación

2

📁 Necesitan personal calificado

No Requieren

1

📁 Asignación libre

Empleados Calificados

4

📁 Con al menos una calificación

🔍

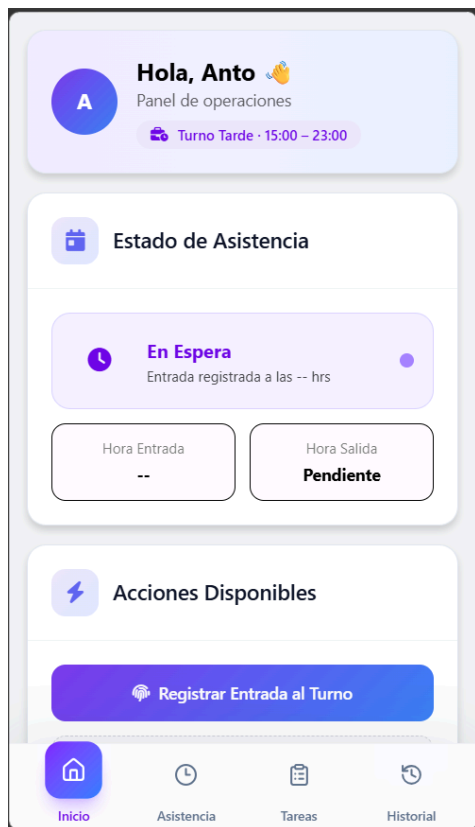
Buscar categoría por nombre...

Calificación: Todas

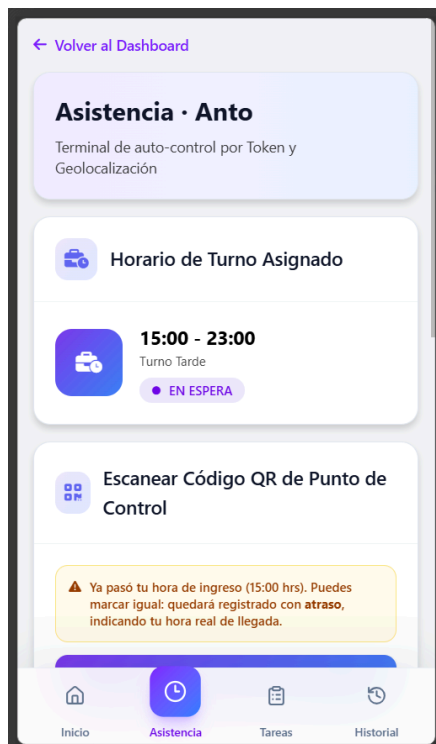
🔼 🔽 A - Z

📁 Categoría	📝 Descripción	🌟 Calificación Requerida	Estado	Acciones
Gestión de Bodega	Control y gestión de inventario y bodega	Si requiere	Activa	👤 📝 🗑️
Operación de Maquinaria	Manejo y operación de maquinaria agrícola pesada	Si requiere	Activa	👤 📝 🗑️
Trabajo de Campo	Actividades generales de campo como siembra, riego y cosecha	No requiere	Activa	📝 🗑️

Dashboard del empleado (EmployeeDashboard): Vista simplificada para empleados de terreno. Muestra el turno activo, las tareas del día y el estado de asistencia. Acceso mediante EmployeeLayout con navegación reducida.



Asistencia del empleado (EmployeeAsistencia): Interfaz para registro de asistencia mediante ingreso de token de 4 caracteres o escaneo de código QR generado por el encargado. Incluye indicador de estado del token (vigente/expirado).



Tareas del empleado (EmployeeTareas): Vista de tareas asignadas al empleado para el día activo, con opciones para marcar inicio y finalización de cada tarea.



Historial del empleado (EmployeeHistorial): Historial personal de asistencias y tareas completadas, con filtros por rango de fechas.



Capítulo IV. Desarrollo

IV.1 Diseño de arquitectura

El sistema de administración de los módulos, basado en una arquitectura cliente-servidor como se mencionó anteriormente, se desarrolla con el stack PERN para garantizar el control de los procesos y la seguridad de la información de los usuarios y la empresa. Esta arquitectura separa claramente la interfaz de usuario, para así facilitar el mantenimiento y la actualización del sistema.

Backend (Servidor): Se implementa una API REST desarrollada con Node.js y Express, encargada de gestionar la lógica de negocio, los procesos de autenticación y datos relacionados con los módulos del sistema (proyectos, actividades, personal, etc.).

Frontend (Cliente): La interfaz se construye con REACT y se utiliza Vite como herramienta de desarrollo para la optimización de tiempo de compilación y carga. El diseño visual se realiza con Tailwind CSS, lo que permite crear una interfaz moderna, consistente y adaptable a cualquier dispositivo.

Base de datos: Se emplea PostgreSQL como sistema gestor de bases de datos relacional, respaldado por TypeORM como ORM para el mapeo de entidades y relaciones. Esta composición asegura la integridad de los datos mediante migraciones controladas y validaciones a nivel esquema, así, garantiza la consistencia de información gestionada por cada módulo del sistema.

IV.2 Estructura del código

IV.2.1 Backend

Directorio	Detalle
src	Directorio donde se almacenan el resto de subcarpetas con lógica de Backend.
config	Contiene los archivos de configuración general de la aplicación, como la conexión a la base de datos, variables de entorno, passport y datos base o configuraciones auxiliares.
entity	Define las entidades del modelo de datos, es decir, las tablas o modelos de la base de datos y sus atributos.
handlers	Agrupar utilidades para manejar respuestas estándar de la API, como respuestas exitosas, errores y formatos consistentes.
middlewares	Almacena funciones intermedias que se ejecutan antes de llegar al controlador, por ejemplo autenticación, autorización y validación de roles.
modules	Organiza la lógica de negocio por dominio o funcionalidad, con subcarpetas como usuarios, turnos, asistencia, proyectos y tareas, donde cada una posee su propio service, controller, routes y validation.
routes	Define las rutas HTTP de la API y las asocia con los controladores correspondientes.
seed	Contiene datos iniciales y scripts para poblar la base de datos con información base para pruebas o desarrollo.
shared	Incluye utilidades reutilizables compartidas por varias partes del backend, como validadores, helpers de base de datos y funciones comunes.
app.js	Punto de configuración principal de la aplicación Express. Aquí se montan middlewares, rutas y la configuración general del servidor.
index.js	Archivo de arranque del backend. Se encarga de iniciar el servidor y conectar la aplicación con la base de datos.

IV.2.2 Frontend

Directorio	Detalle
src	Directorio donde se almacenan el resto de subcarpetas con lógica de Frontend.
api	Contiene los servicios que consumen la API del backend.
assets	Recursos estáticos como imágenes, iconos y otros archivos visuales.
components	Componentes reutilizables de la interfaz, como botones, modales, tablas y barras laterales.
context	Contexto global de React para manejar estado compartido, como autenticación y color de tema.
data	Datos estáticos o configuraciones usadas por la aplicación.
layouts	Estructuras visuales reutilizables de páginas, como plantillas de contenido o formularios.
pages	Vistas principales de la aplicación organizadas por módulos o roles.
routes	Definición de rutas y navegación de la aplicación.
styles	Estilos generales o hojas de estilo compartidas.
utils	Funciones auxiliares y utilidades reutilizables.
App.jsx	Componente principal de la interfaz. Define la estructura general de la app y suele contener la configuración de rutas o layouts.
index.css	Hoja de estilos globales de la aplicación.
main.jsx	Punto de entrada de la aplicación React. Se encarga de renderizar el componente raíz en el DOM.

Capítulo V Conclusión del proyecto

Al comenzar este proyecto, el principal desafío no fue tecnológico, sino de comprensión: entender cómo funciona realmente una empresa de aseo industrial, con equipos dispersos en distintas instalaciones y una operación que depende casi por completo del control manual. Solo después de conversar con la empresa y revisar cómo se maneja hoy la asistencia, el inventario y la asignación de tareas, quedó claro por qué estas actividades generaban tantos problemas: no existía ningún sistema que conectará esa información.

CleanAdmin surge como respuesta a eso. No pretende ser una solución que resuelva todo de una vez, sino una base sólida que ataca los puntos más críticos detectados durante el levantamiento: el marcaje de asistencia poco confiable, la falta de trazabilidad en bodega y la asignación de tareas sin considerar si el trabajador estaba realmente calificado para ejecutarlas. El uso de tokens y QR para el registro de entrada, por ejemplo, buscó cerrar la puerta al marcaje por terceros, algo que en los métodos en papel era prácticamente imposible de controlar. Lo mismo ocurre con el módulo de bodega, donde cada solicitud queda registrada y debe pasar por aprobación, así evita que el stock se mueva sin dejar rastro.

Un aspecto que tomó bastante trabajo definir fue el sistema de roles. No bastaba con decir "el administrador puede todo y el empleado casi nada"; hubo que pensar con detalle qué puede hacer un jefe que no pueda hacer un supervisor, o por qué ciertas acciones deben quedar reservadas solo para Root. Esa jerarquía terminó siendo uno de los pilares del sistema, porque sin ella la información sensible de la empresa quedaría expuesta a cualquiera con acceso a la plataforma.

En cuanto al desarrollo, se optó por el stack PERN (PostgreSQL, Express, React y Node.js) principalmente porque permite separar bien las responsabilidades entre backend y frontend, algo que se agradece cuando el equipo trabaja de forma distribuida. TypeORM ayudó bastante a mantener el orden en el modelo de datos, sobre todo considerando la cantidad de entidades relacionadas que requiere un sistema de este tipo. También se buscó cuidar aspectos que muchas veces se dejan de lado en proyectos académicos, como el hashing de contraseñas, el uso de JWT o el borrado lógico en vez de eliminar información histórica.

Más allá del resultado técnico, este proyecto sirvió para poner en práctica lo que normalmente se ve solo en teoría: pasar de un problema real, contado por alguien que lo vive día a día, a una solución concreta que tiene que funcionar y no solo verse bien en un diagrama. Quedan cosas pendientes, por supuesto. La geolocalización por IP es un buen punto de partida, pero no reemplaza un GPS real; tampoco se abordó el tema de liquidaciones de sueldo, que quedó fuera del alcance a propósito para no desviar el foco del proyecto. Ambos puntos son candidatos naturales para una segunda etapa.

En definitiva, **CleanAdmin** no soluciona todos los problemas de la empresa, pero sí ordena lo que hoy se maneja de forma dispersa y, sobre todo, deja una base sobre la cual se puede seguir construyendo.

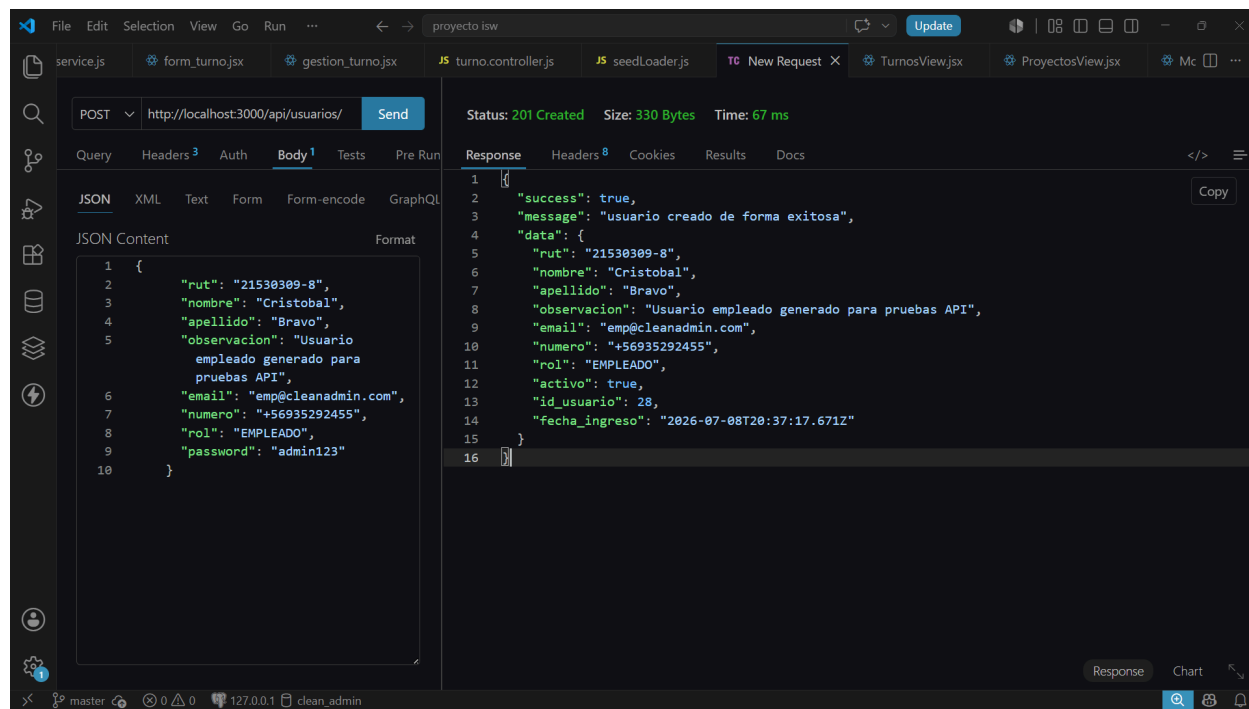
Capítulo VI Bibliografía

Anexos de anexo del Instrumento y datos recopilados de análisis del contexto

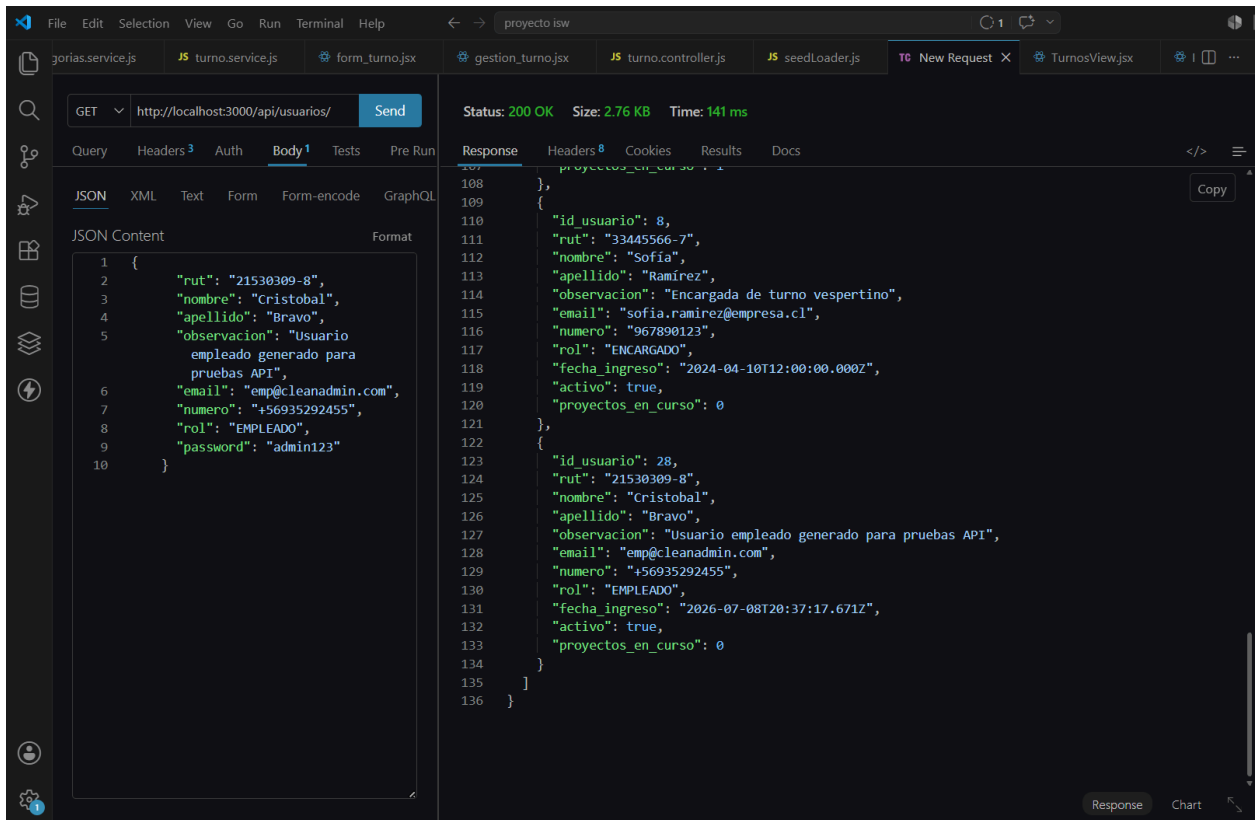
Anexo Pruebas de API:

Pruebas API

Registro de Usuario a través de ThunderClient:



GET usuario



The screenshot shows a REST client interface in VS Code. The request is a GET to `http://localhost:3000/api/usuarios/`. The response is a 200 OK status with a size of 2.76 KB and a time of 141 ms. The response body is a JSON array of two user objects.

```

{
  "rut": "21530309-8",
  "nombre": "Cristobal",
  "apellido": "Bravo",
  "observacion": "Usuario empleado generado para pruebas API",
  "email": "emp@cleanadmin.com",
  "numero": "+56935292455",
  "rol": "EMPLEADO",
  "password": "admin123"
}

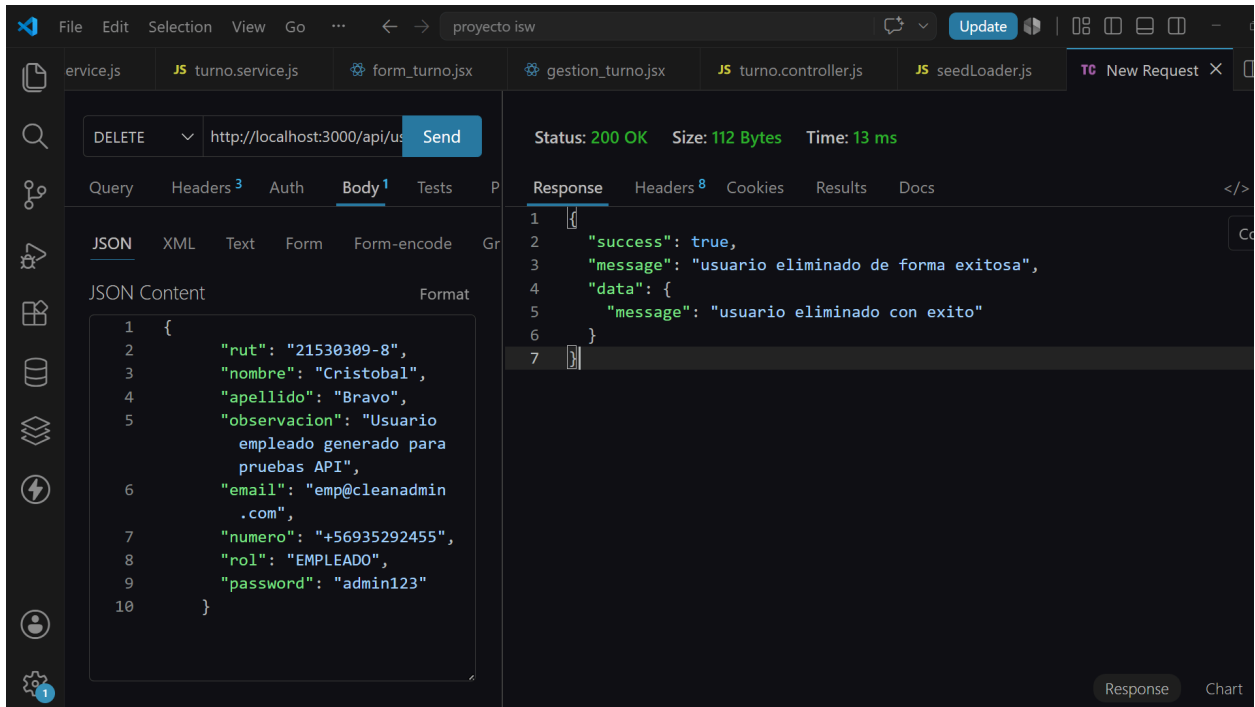
```

```

{
  "id_usuario": 8,
  "rut": "33445566-7",
  "nombre": "Sofia",
  "apellido": "Ramírez",
  "observacion": "Encargada de turno vespertino",
  "email": "sofia.ramirez@empresa.cl",
  "numero": "967890123",
  "rol": "ENCARGADO",
  "fecha_ingreso": "2024-04-10T12:00:00.000Z",
  "activo": true,
  "proyectos_en_curso": 0
}

```

DELETE usuario (soft delete)



The screenshot shows a REST client interface in VS Code. The request is a DELETE to `http://localhost:3000/api/usuarios/`. The response is a 200 OK status with a size of 112 Bytes and a time of 13 ms. The response body is a JSON object indicating a successful deletion.

```

{
  "rut": "21530309-8",
  "nombre": "Cristobal",
  "apellido": "Bravo",
  "observacion": "Usuario empleado generado para pruebas API",
  "email": "emp@cleanadmin.com",
  "numero": "+56935292455",
  "rol": "EMPLEADO",
  "password": "admin123"
}

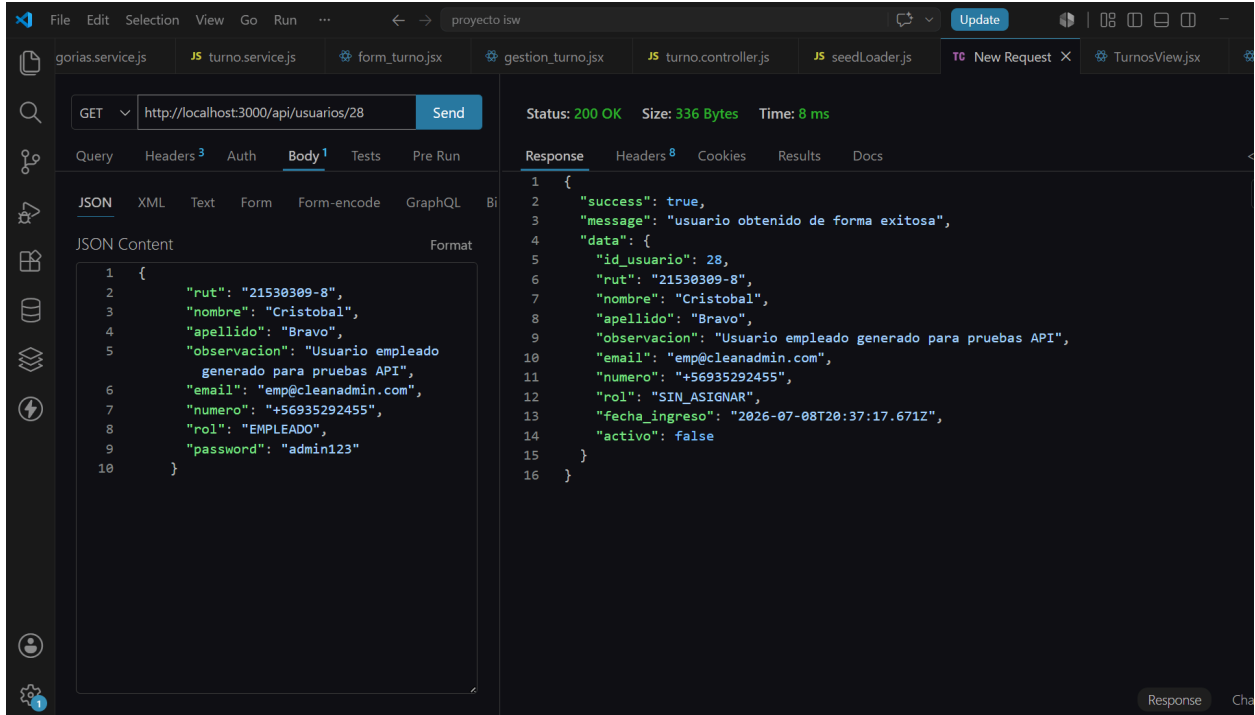
```

```

{
  "success": true,
  "message": "usuario eliminado de forma exitosa",
  "data": {
    "message": "usuario eliminado con exito"
  }
}

```

GET luego de eliminar



The screenshot shows a REST client interface in VS Code. The request is a GET to `http://localhost:3000/api/usuarios/28`. The response is a 200 OK status with a JSON body containing user details.

```
GET http://localhost:3000/api/usuarios/28
```

JSON Content:

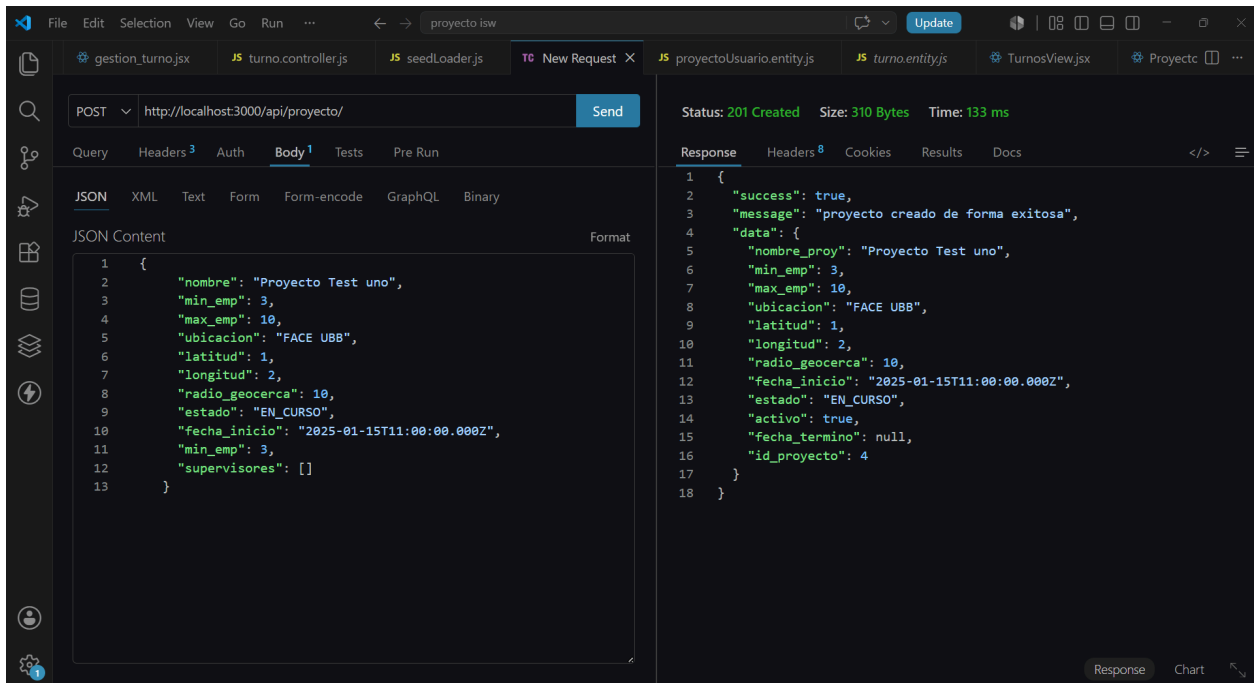
```
1 {
2   "rut": "21530309-8",
3   "nombre": "Cristobal",
4   "apellido": "Bravo",
5   "observacion": "Usuario empleado
6     generado para pruebas API",
7   "email": "emp@cleanadmin.com",
8   "numero": "+56935292455",
9   "rol": "EMPLEADO",
10  "password": "admin123"
11 }
```

Response:

```
1 {
2   "success": true,
3   "message": "usuario obtenido de forma exitosa",
4   "data": {
5     "id_usuario": 28,
6     "rut": "21530309-8",
7     "nombre": "Cristobal",
8     "apellido": "Bravo",
9     "observacion": "Usuario empleado generado para pruebas API",
10    "email": "emp@cleanadmin.com",
11    "numero": "+56935292455",
12    "rol": "SIN_ASIGNAR",
13    "fecha_ingreso": "2026-07-08T20:37:17.671Z",
14    "activo": false
15  }
16 }
```

Sigue el usuario presente en la base de datos, pero se mantiene inactivo debido al soft delete anterior.

POST proyecto



The screenshot shows a REST client interface in VS Code. The request is a POST to `http://localhost:3000/api/proyecto/`. The response is a 201 Created status with a JSON body containing project details.

```
POST http://localhost:3000/api/proyecto/
```

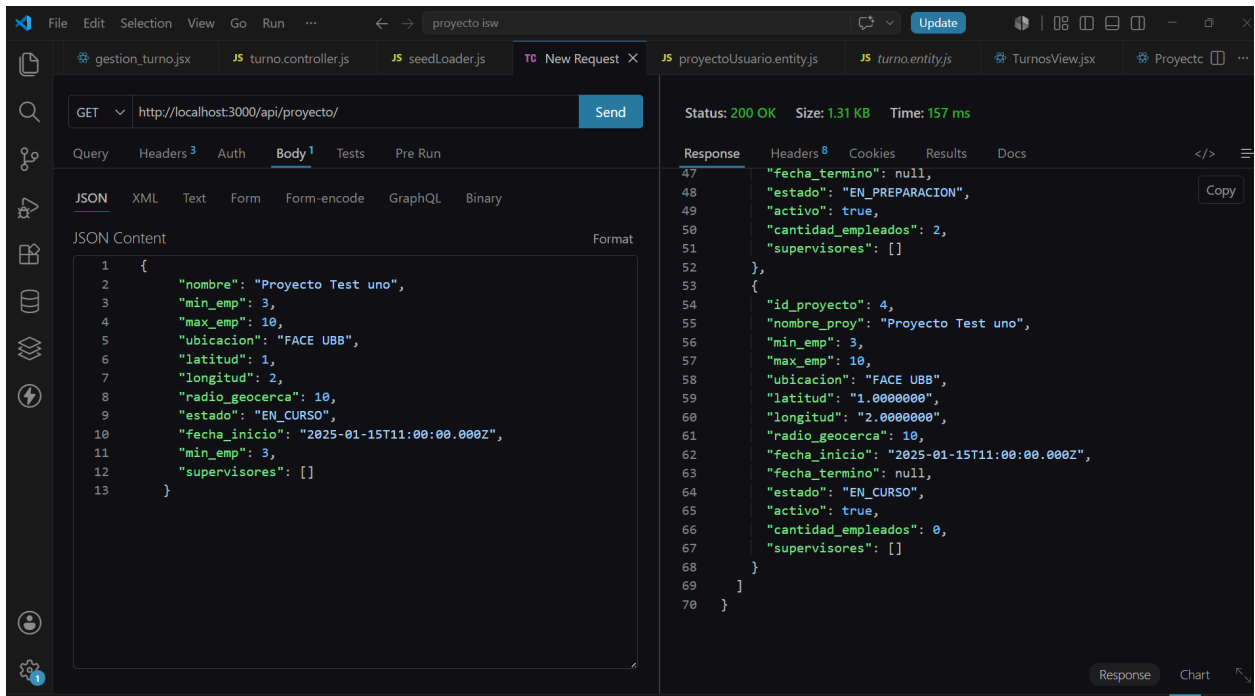
JSON Content:

```
1 {
2   "nombre": "Proyecto Test uno",
3   "min_emp": 3,
4   "max_emp": 10,
5   "ubicacion": "FACE UBB",
6   "latitud": 1,
7   "longitud": 2,
8   "radio_geocerca": 10,
9   "estado": "EN_CURSO",
10  "fecha_inicio": "2025-01-15T11:00:00.000Z",
11  "min_emp": 3,
12  "supervisores": []
13 }
```

Response:

```
1 {
2   "success": true,
3   "message": "proyecto creado de forma exitosa",
4   "data": {
5     "nombre_proy": "Proyecto Test uno",
6     "min_emp": 3,
7     "max_emp": 10,
8     "ubicacion": "FACE UBB",
9     "latitud": 1,
10    "longitud": 2,
11    "radio_geocerca": 10,
12    "fecha_inicio": "2025-01-15T11:00:00.000Z",
13    "estado": "EN_CURSO",
14    "activo": true,
15    "fecha_termino": null,
16    "id_proyecto": 4
17  }
18 }
```

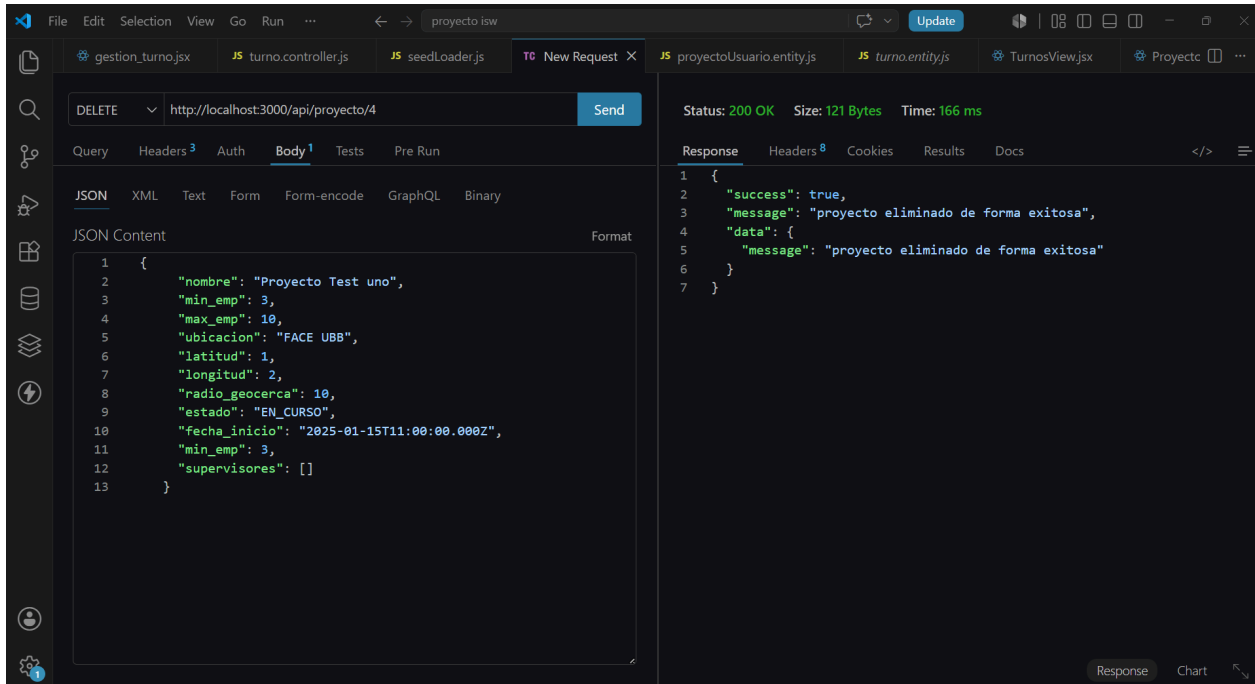
Get proyecto



The screenshot shows the VS Code REST Client interface. The request is a GET to `http://localhost:3000/api/proyecto/`. The response is a 200 OK with a size of 1.31 KB and a time of 157 ms. The response body is a JSON object:

```
47 {
48   "fecha_termino": null,
49   "estado": "EN_PREPARACION",
50   "activo": true,
51   "cantidad_empleados": 2,
52   "supervisores": []
53 },
54 {
55   "id_proyecto": 4,
56   "nombre_proy": "Proyecto Test uno",
57   "min_emp": 3,
58   "max_emp": 10,
59   "ubicacion": "FACE UBB",
60   "latitud": "1.0000000",
61   "longitud": "2.0000000",
62   "radio_geocerca": 10,
63   "fecha_inicio": "2025-01-15T11:00:00.000Z",
64   "min_emp": 3,
65   "supervisores": []
66 }
67 ]
68 }
69 ]
70 }
```

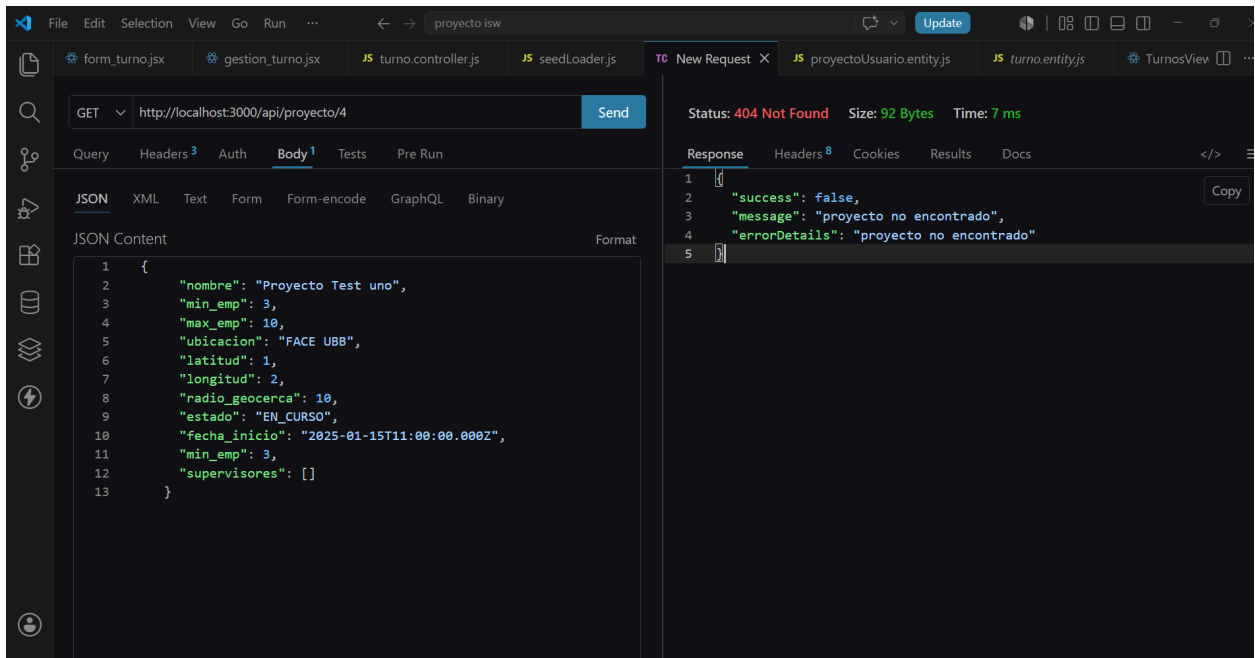
DELETE proyecto (hard delete)



The screenshot shows the VS Code REST Client interface. The request is a DELETE to `http://localhost:3000/api/proyecto/4`. The response is a 200 OK with a size of 121 Bytes and a time of 166 ms. The response body is a JSON object:

```
1 {
2   "success": true,
3   "message": "proyecto eliminado de forma exitosa",
4   "data": {
5     "message": "proyecto eliminado de forma exitosa"
6   }
7 }
```

GET proyecto (luego de hard delete)



GET http://localhost:3000/api/proyecto/4

Status: 200 OK Size: 92 Bytes Time: 7 ms

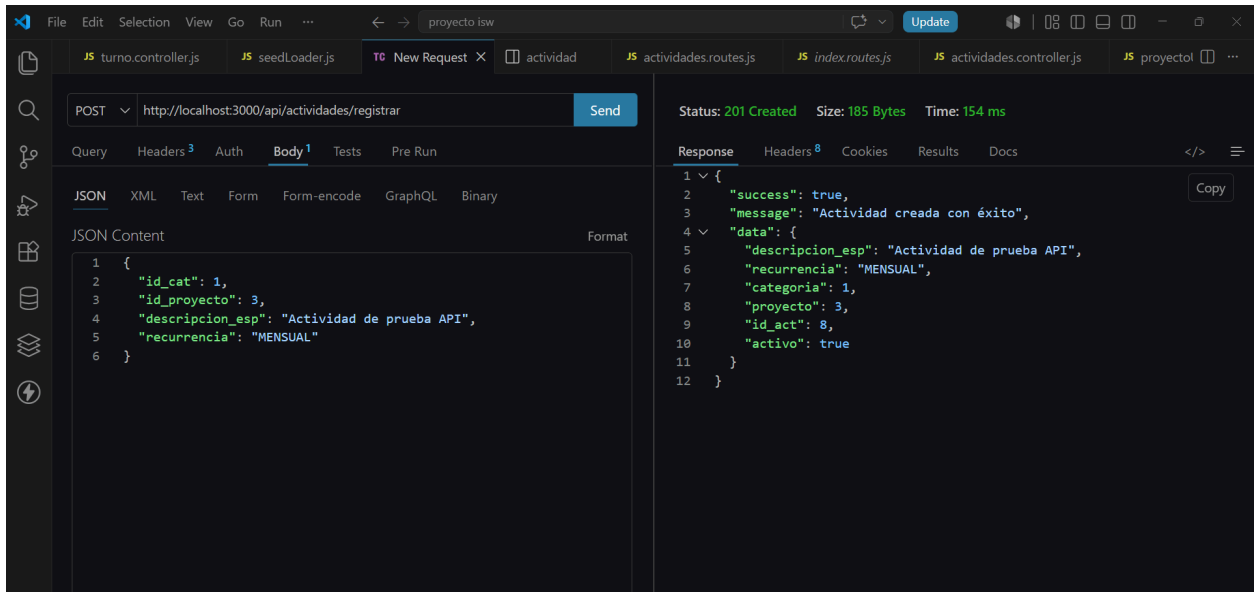
Response

```
1 {
2   "success": true,
3   "message": "Proyecto encontrado",
4   "errorDetails": ""
5 }
```

JSON Content

```
1 {
2   "nombre": "Proyecto Test uno",
3   "min_emp": 3,
4   "max_emp": 10,
5   "ubicacion": "FACE UBB",
6   "latitud": 1,
7   "longitud": 2,
8   "radio_geocerca": 10,
9   "estado": "EN_CURSO",
10  "fecha_inicio": "2025-01-15T11:00:00.000Z",
11  "min_emp": 3,
12  "supervisores": []
13 }
```

POST registro de actividad



POST http://localhost:3000/api/actividades/regarstrar

Status: 201 Created Size: 185 Bytes Time: 154 ms

Response

```
1 {
2   "success": true,
3   "message": "Actividad creada con éxito",
4   "data": {
5     "descripcion_esp": "Actividad de prueba API",
6     "recurrencia": "MENSUAL",
7     "categoria": 1,
8     "proyecto": 3,
9     "id_act": 8,
10    "activo": true
11  }
12 }
```

JSON Content

```
1 {
2   "id_cat": 1,
3   "id_proyecto": 3,
4   "descripcion_esp": "Actividad de prueba API",
5   "recurrencia": "MENSUAL"
6 }
```

GET actividades



projecto isw

JS turno.controller.js JS seedLoader.js TC New Request X actividad JS actividades.routes.js JS index.routes.js JS actividades.controller.js JS proyectol

GET http://localhost:3000/api/actividades/ Send

Status: 200 OK Size: 3.99 KB Time: 138 ms

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id_cat": 1,
3   "id_proyecto": 3,
4   "descripcion_esp": "Actividad de prueba API",
5   "recurrencia": "MENSUAL"
6 }
```

Response Headers 8 Cookies Results Docs

```
1 {
2   "success": true,
3   "message": "Catálogo de actividades obtenido",
4   "data": [
5     {
6       "id_act": 1,
7       "descripcion_esp": "Revisión y mantenimiento del sistema de
8         riego por goteo",
9       "recurrencia": "SEMANAL",
10      "activo": true,
11      "categoria": {
12        "id_cat": 2,
13        "nombre": "Trabajo de Campo",
14        "descripcion": "Actividades generales de campo como siembra,
15          riego y cosecha",
16        "requiere_calificacion": false,
17        "activo": true
18      },
19      "proyecto": {
20        "id_proyecto": 1,
21        "nombre_proy": "Proyecto Riego Norte",
22        "min_emp": 3,
23        "max_emp": 10,
24        "ubicacion": "Sector Norte, Km 45, Ruta 5",
25        "latitud": null,
26        "longitud": null,

```